This Module Introduction page features a prominent banner image across the top and a numbered list below.

This page can easily accommodate more content than the Course Introduction page. And, as illustrated below, the numbered list can be used to list a module's learning objectives. This makes it ideal as the first page of a module in a course.

Recommended banner image size: at least 1200 pixels wide and 400 pixels high.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Recall the basic principles cloud-native app scaling.

02 Define the methodology applied for cloud-native app scaling.

Learning Activity



Match the scaling concepts with their corresponding descriptions. This activity should take approximately 5 minutes to complete.

Learning Activity Module 9

1. Horizontal Scaling
2. Vertical Scaling
3. Auto-Scaling
4. Stateless Applications
5. Load Balancing

A. Adding or removing resou such as CPU or memory, to an e server.

B. Distributing incoming net traffic across multiple server ensure no single server is overwhelmed.

C. Adjusting the number of se or resources in response to app demand.

D. Increasing the number of s to handle a greater workloa

E. Applications that do not s session data, making them eas scale.

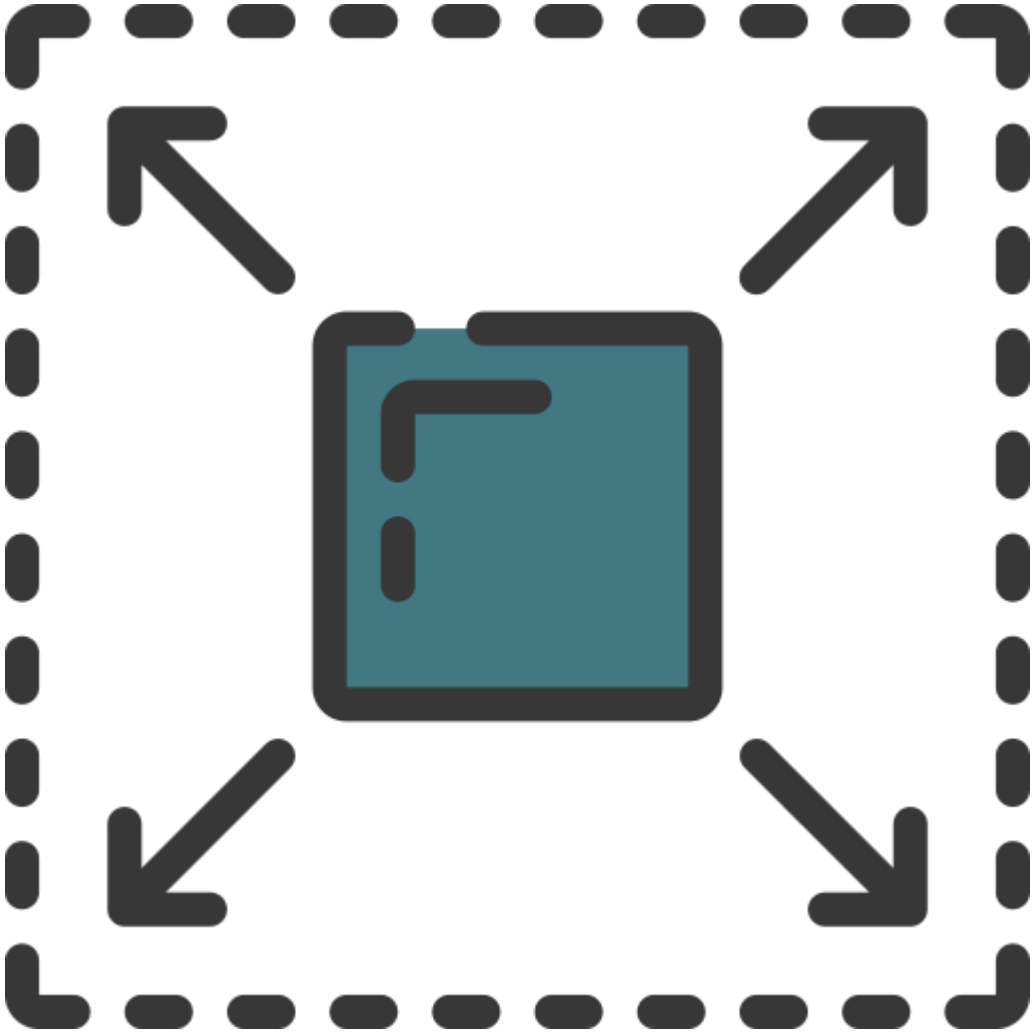
Check

H5P Reuse I am confused

Scaling cloud-native applications

One of the strongest advantages of moving to a cloud hosting environment is scalability.

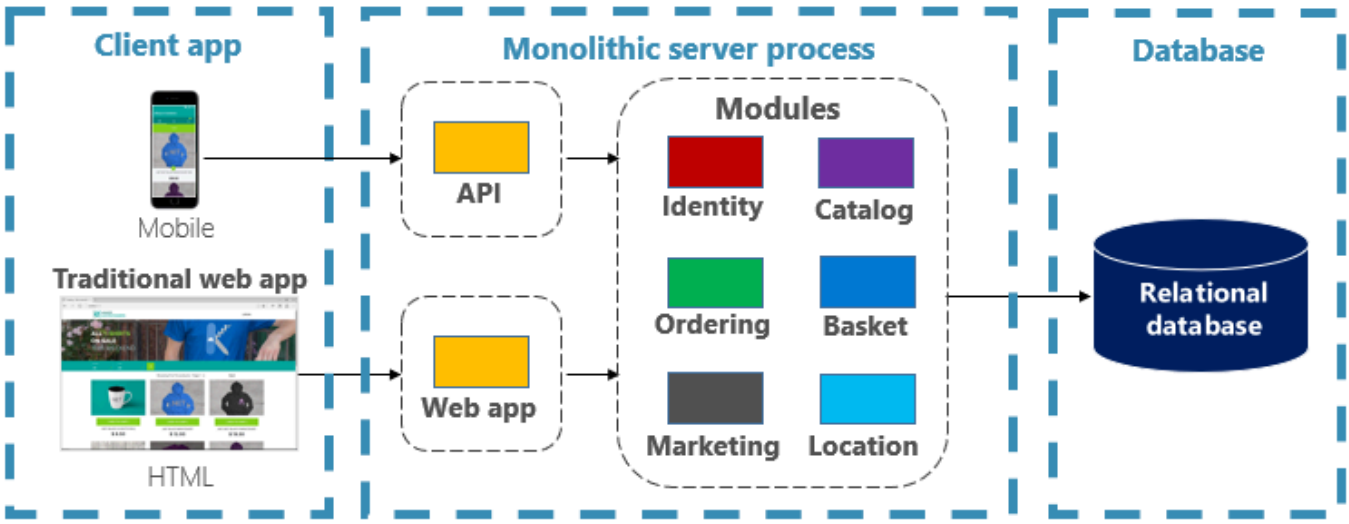
Scalability, or the ability for an application to accept additional user load without compromising performance for each user. It's most often achieved by breaking up an application into small pieces that can each be given whatever resources they require.



Cloud vendors enable massive scalability anytime and anywhere in the world.

Challenges with monolithic deployments

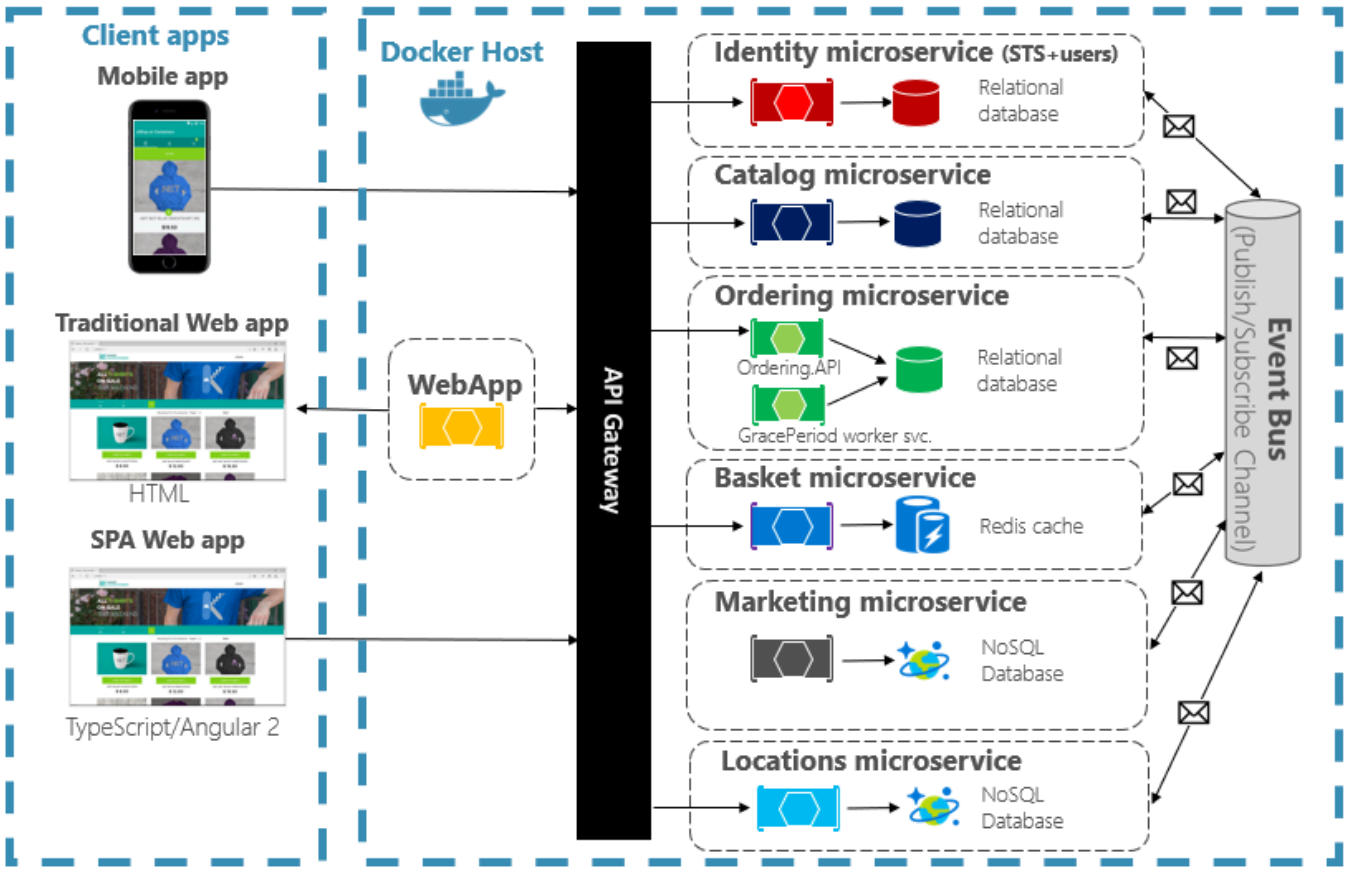
You will recall from earlier lessons that a monolithic application is hosted entirely on a single machine instance, often requiring high-capability hardware. If any part of the monolith requires scaling, another copy of the entire application must be deployed to another machine. With a monolith, you can't scale application components individually - it's all or nothing. Scaling components that don't require scaling results in inefficient and costly resource usage.



Embracing microservices

Early in the course, we discussed how cloud-native app development uses independent and isolated microservices to provide the functionalities that the application requires.

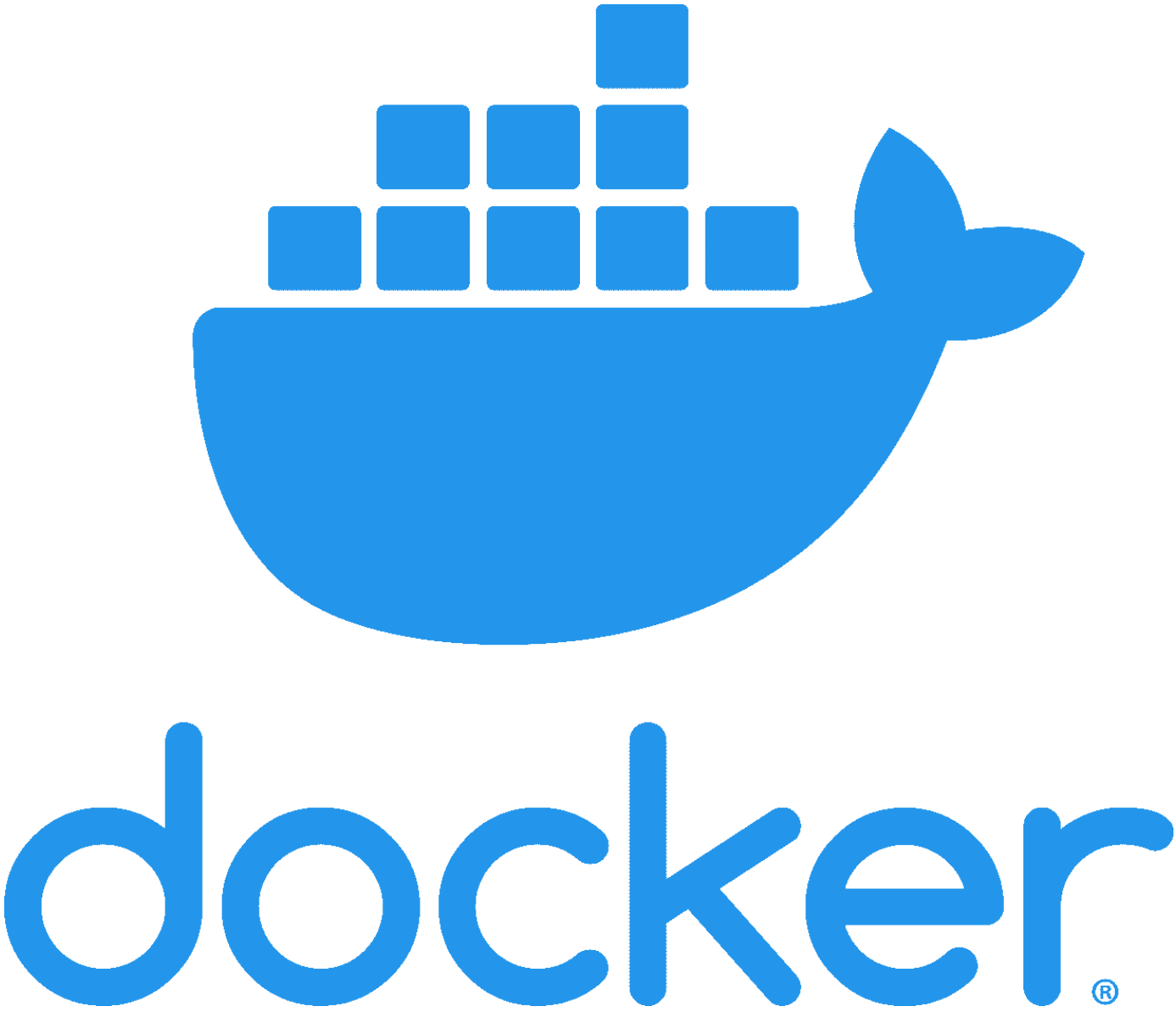
This leads an architecture that is distributed and takes advantages of containerization. A monolithic application can be gradually broken down into microservices by delegating certain features to microservices. The remaining monolithic application can then itself be containerized.



Why containerize microservices?

Containers are immutable. Once you define a container, you can recreate and run it exactly the same way. This immutability lends itself to component-based design.

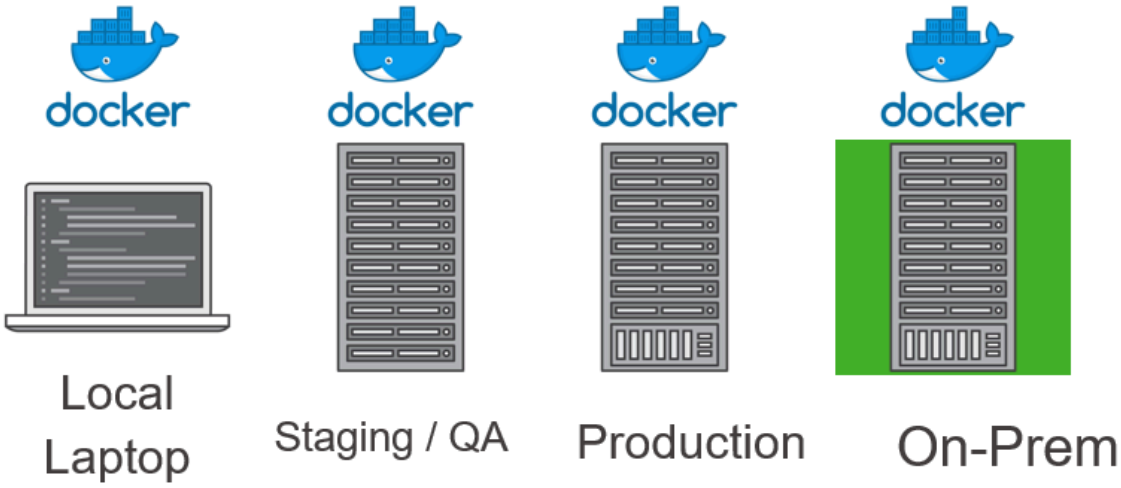
If some parts of an application evolve differently than others, why redeploy the entire app when you can just deploy the parts that change most frequently? Different features and cross-cutting concerns of an app can be broken up into separate units.



Container & Docker Benefits

Containerization creates portable application artifacts that runs reliably everywhere. It allows to run different applications or application versions with different dependencies simultaneously.

Better resource utilization can be achieved by running multiple lightweight containers per host, instead of dedicating the entire operating system of a VM to running a microservice.



Cloud-native services

Each cloud-native service is built and deployed in a separate container, and each service can be updated as needed, independently from the other services.

Individual services can be hosted on nodes with resources appropriate to each service. The environment each service runs in is immutable, shared across dev, test, and production environments, and easily versioned.

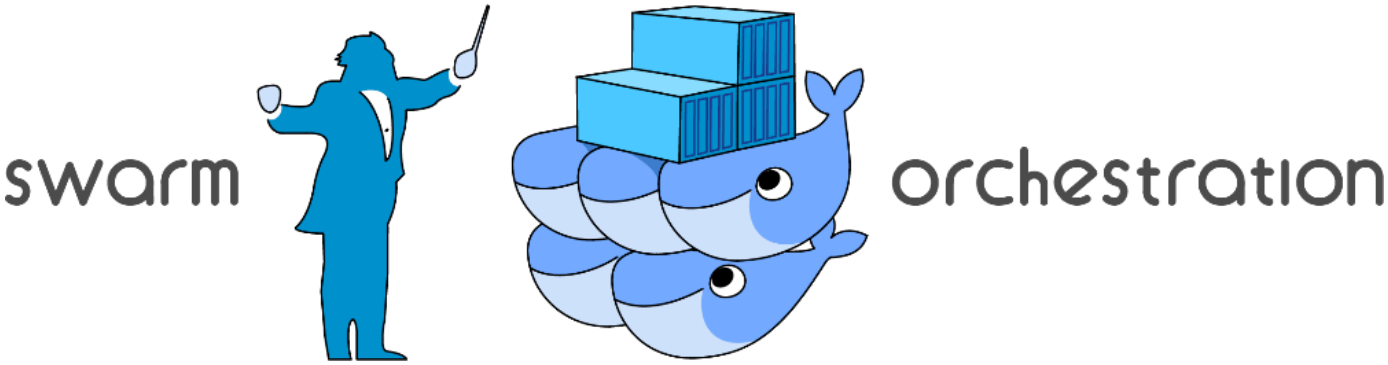
Coupling between different areas of the application occurs explicitly as calls or messages between services, not compile-time dependencies within the monolith. You can also choose the technology that best suites a given capability without requiring changes to the rest of the app.

Automated Management

Containerized services require automated management. It wouldn't be feasible to manually administer a large set of independently deployed containers. For example, consider the following tasks:

- How will container instances be provisioned across a cluster of many machines?
- Once deployed, how will containers discover and communicate with each other?
- How can containers scale in or out on-demand?
- How do you monitor the health of each container?

- How do you protect a container against hardware and software failures?
- How do upgrade containers for a live application with zero downtime?
  As we covered in previous lessons, container orchestration can provide the automation required to manage these tasks. Recall that:
- While tools such as Docker create images and run containers, you also need tools to manage them.
- Container management is done with a special software program called a container orchestrator.
- When operating at scale with many independent running containers, orchestration is essential.
  A tool such as Kubernetes allows this orchestration to be configured and deployed through automation.



Orchestration and scaling

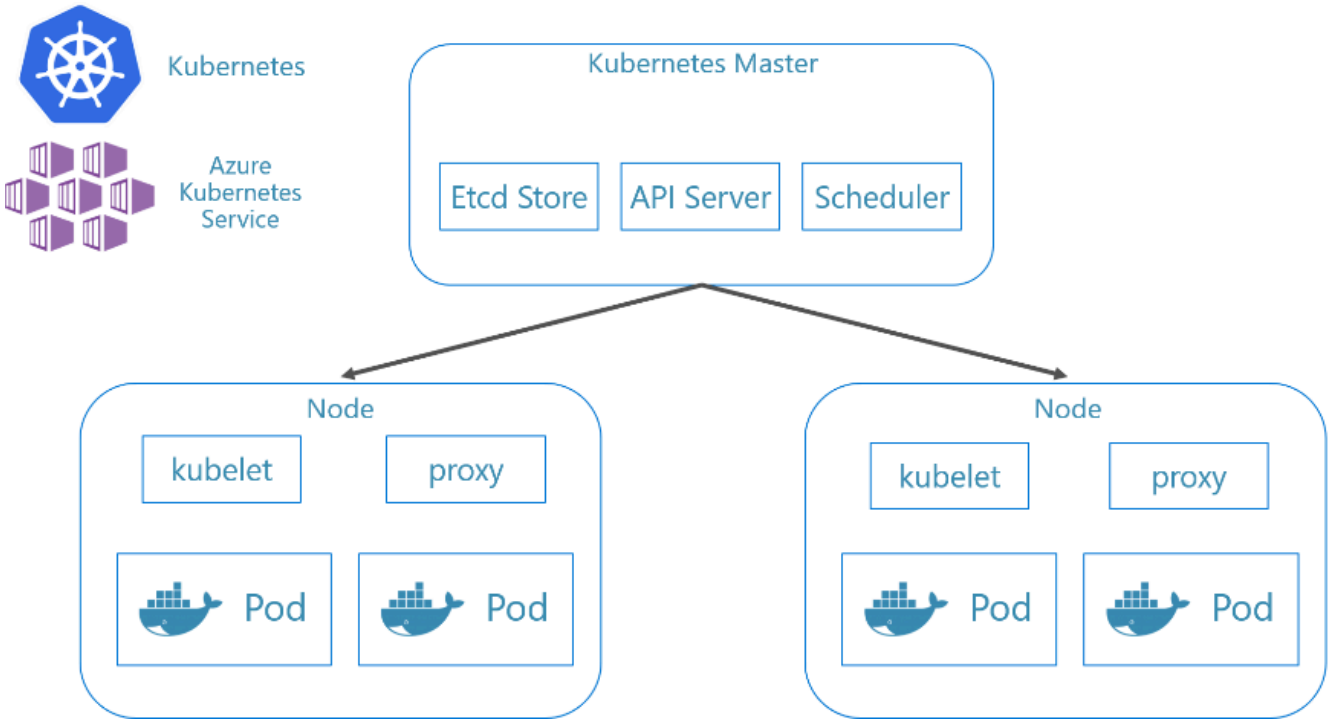
Services built on containers can leverage scaling benefits provided by orchestration tools like Kubernetes.

By design containers only know about themselves. Once you have multiple containers that need to work together, you should organize them at a higher level.

Organizing large numbers of containers and their shared dependencies, such as network configuration, is where orchestration tools come in to save the day!

What are the scaling benefits?

Kubernetes creates an abstraction layer over groups of containers and organizes them into pods. Pods run on worker machines referred to as nodes. This organized structure is referred to as a cluster.



Kubernetes supports both declarative and imperative configuration. The imperative approach involves running various commands that tell Kubernetes what to do each step of the way, for instance:

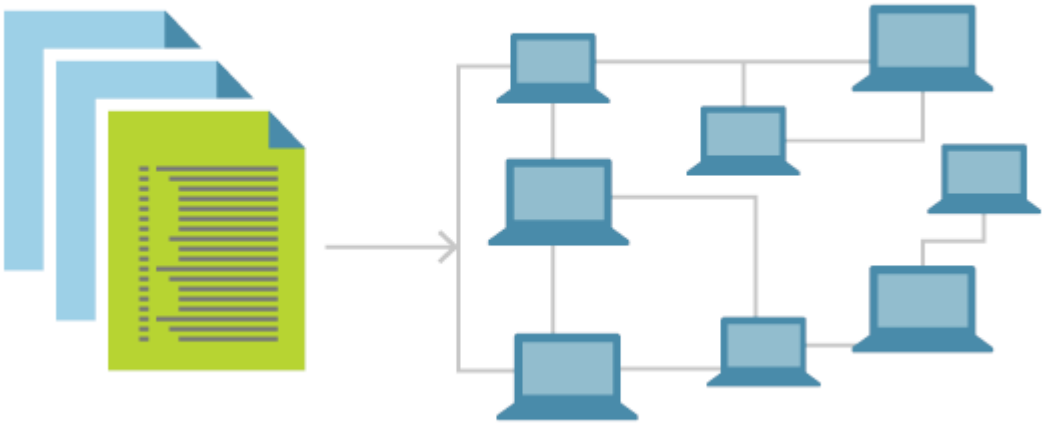
- Run this image.
- Delete this pod.
- Expose this port.

With the declarative approach, you create a configuration file, called a manifest, to describe what you want instead of what to do. Kubernetes reads the manifest and transforms your desired end state into actual end state.

Declarative versus Imperative

Imperative commands are great for learning and interactive experimentation. However, you'll want to declaratively create Kubernetes manifest files to embrace an infrastructure as code approach, providing for reliable and repeatable deployments.

The manifest file becomes a project artifact and is used in your CI/CD pipeline for automating Kubernetes deployments. Using declarative configuration allows infrastructure to be represented as code that can be checked in and versioned alongside the application code. It provides improved change control and better support for continuous deployment using a build and deploy pipeline.



Scenarios for containers and orchestrators

In this section, we will observe different scenarios where containers and orchestrators can provide an advantage, and when they are not a good fit.



Applications requiring high uptime and scalability

Individual applications that have high uptime and scalability requirements are ideal candidates for cloud-native architectures using microservices, containers, and orchestrators. They can be developed in containers, tested across versioned environments, and deployed into production with zero downtime.

The use of Kubernetes clusters ensures such apps can also scale on demand and recover automatically from node failures.

Large numbers of applications

Organizations that deploy and maintain large numbers of applications benefit from containers and orchestrators. The upfront effort of setting up containerized environments and Kubernetes clusters is primarily a fixed cost.

Deploying, maintaining, and updating individual applications has a cost that varies with the number of applications. Beyond a few applications, the complexity of maintaining custom applications manually exceeds the cost of implementing a solution using containers and orchestrators.

When should you avoid using containers and orchestrators?

If you're unable to build your application following the Twelve-Factor App principles, you should consider avoiding containers and orchestrators.

In these cases, consider a VM-based hosting platform, or possibly some hybrid system. With it, you can always spin off certain pieces of functionality into separate containers or even serverless functions.

Leveraging serverless functions

Serverless is a relatively new service model of cloud computing. It doesn't mean that servers are optional - your code still runs on a server somewhere. The distinction is that the application team no longer concerns itself with managing server infrastructure and defers to the cloud vendor for this responsibility.

The development team increases its productivity by delivering business solutions to customers, not infrastructure.

Serverless computing uses event-triggered stateless containers to host your services. They can scale out and in to meet demand as needed.

Serverless platforms like Azure Functions have tight integration with other Azure services like queues, events, and storage.



What challenges are solved by serverless?

Serverless platforms address many time-consuming and expensive concerns:

- Purchasing machines and software licenses.
- Housing, securing, configuring, and maintaining the machines and their networking, power, and A/C requirements.
- Patching and upgrading operating systems and software.
- Configuring web servers or machine services to host application software.
- Configuring application software within its platform.



Many companies allocate large budgets to support hardware infrastructure concerns. Moving to the cloud can help reduce these costs; shifting applications to serverless can help eliminate them.

Microservice vs a Serverless Function

Typically, a microservice encapsulates a business capability, such as a shopping cart for an online eCommerce site. It exposes multiple operations that enable a user to manage their shopping experience.

A function, however, is a small, lightweight block of code that executes a single-purpose operation in response to an event.

Microservices are typically constructed to respond to requests, often from an interface. Requests can be HTTP Rest- or gRPC-based.

Serverless services respond to events. Its event-driven architecture is ideal for processing short-running, background tasks.

What scenarios are appropriate for serverless?

Serverless exposes individual short-running functions that are invoked in response to a trigger. This makes them ideal for processing background tasks.

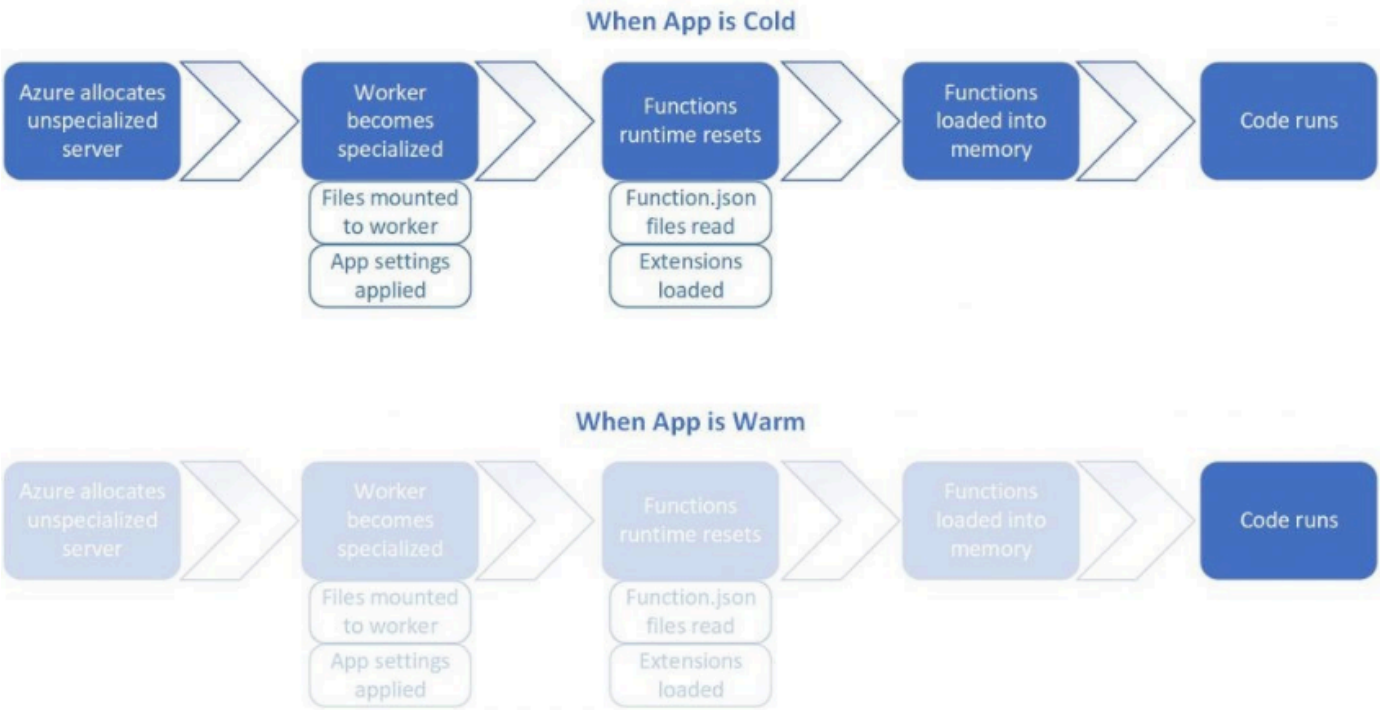
Many services have long-running processes as part of their workflows. Often these tasks are done as part of the user's interaction with the application. These tasks can force the user to wait, negatively impacting their experience.

Serverless computing provides a great way to move slower tasks outside of the user interaction loop. These tasks can scale with demand without requiring the entire application to scale.

When should you avoid serverless?

Serverless solutions provision and scale on demand. When a new instance is invoked, cold starts are a common issue. A cold start is the period of time it takes to provision this instance. Normally, this delay might be a few seconds, but can be longer depending on various factors.

Once provisioned, a single instance is kept alive as long as it receives periodic requests. But, if a service is called less frequently, it may be removed it from memory and require a cold start when reinvoked. Cold starts are also required when a function scales out to a new instance.



Cloud providers bill for serverless based on compute execution time and consumed memory. Long running operations or high memory consumption workloads aren't always the best candidates for serverless.

Serverless functions favor small chunks of work that can complete quickly. Most serverless platforms require individual functions to complete within a few minutes. Azure Functions defaults to a 5-minute time-out duration, which can be configured up to 10 minutes.

Cloud-native communication patterns

When constructing a cloud-native system, communication becomes a significant design decision. Some of the design decisions will include:

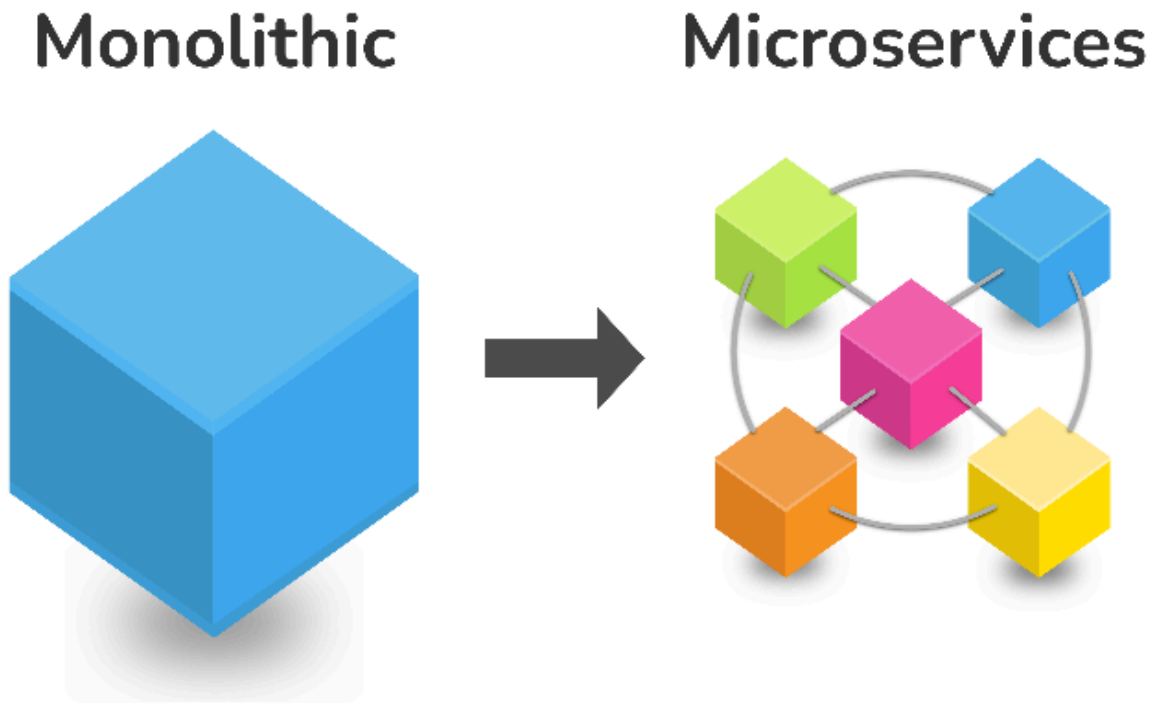
- How does a front-end client application communicate with a backend microservice?
- How do back-end microservices communicate with each other?

In this lesson, we will explore what are the principles, patterns, and best practices to consider when implementing communication in cloud-native applications.

Communication considerations

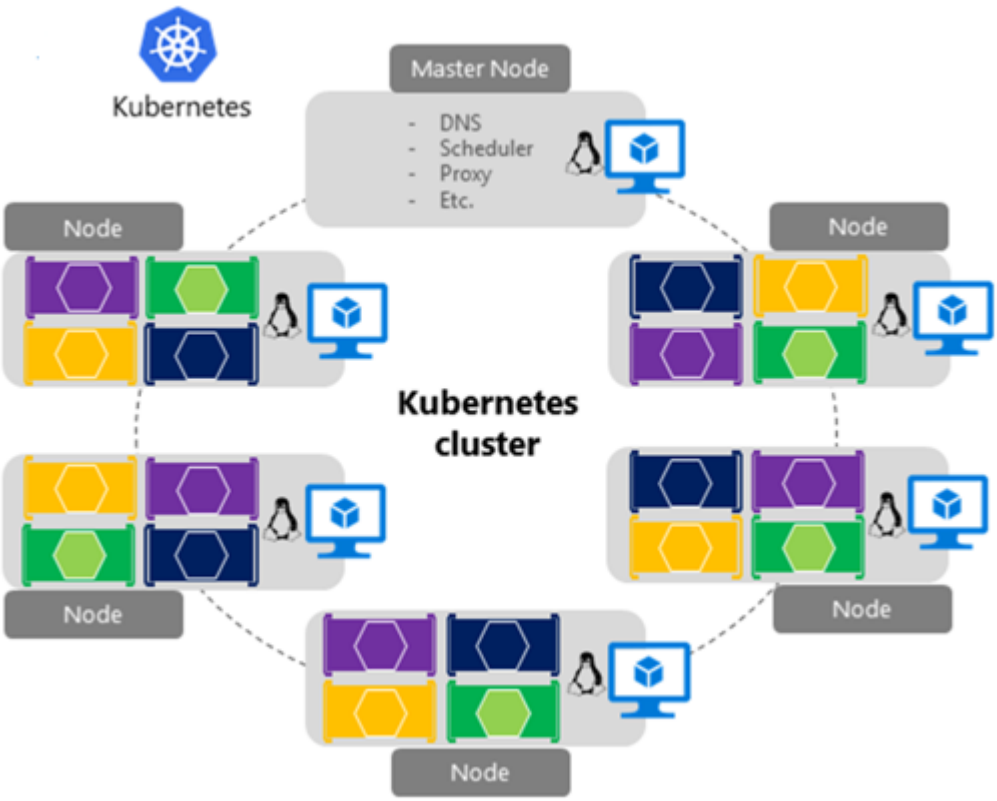
In a monolithic application, communication is straightforward since the code modules execute together in the same executable space (process) on a server. This approach can have performance advantages as everything runs together in shared memory, but results in tightly coupled code that becomes difficult to maintain, evolve, and scale.

On the other hand, cloud-native systems implement a microservice-based architecture with many small, independent microservices. Each microservice executes in a separate process and typically runs inside a container that is deployed to a cluster.



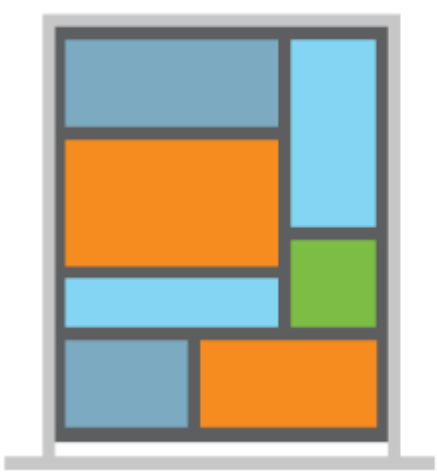
K8S Cluster

A cluster groups a pool of virtual machines together to form a highly available environment. They're managed with an orchestration tool, which is responsible for deploying and managing the containerized microservices.



This figure shows a Kubernetes cluster deployed into the Azure cloud with the fully managed Azure Kubernetes Services.

Across the cluster, microservices communicate with each other through APIs and messaging technologies. While they provide many benefits, microservices have their inconveniences. Local in-process method calls between components are now replaced with network calls, so each microservice must communicate over a network protocol, which adds complexity to your system.





Challenges of microservices communication

Among the challenges experienced when implementing communication call between microservices, we can list:

- Network congestion, latency, and transient faults are a constant concern.
- Resiliency (that is, retrying failed requests) is essential.
- Some calls must be idempotent as to keep consistent state.
- Each microservice must authenticate and authorize calls.
- Each message must be serialized and then deserialized - which can be expensive.
- Message encryption/decryption becomes important.

Microservices Communication

In the next sections, we will address communication between front-end applications and back-end microservices. We'll then look at back-end microservices communicate with each other.

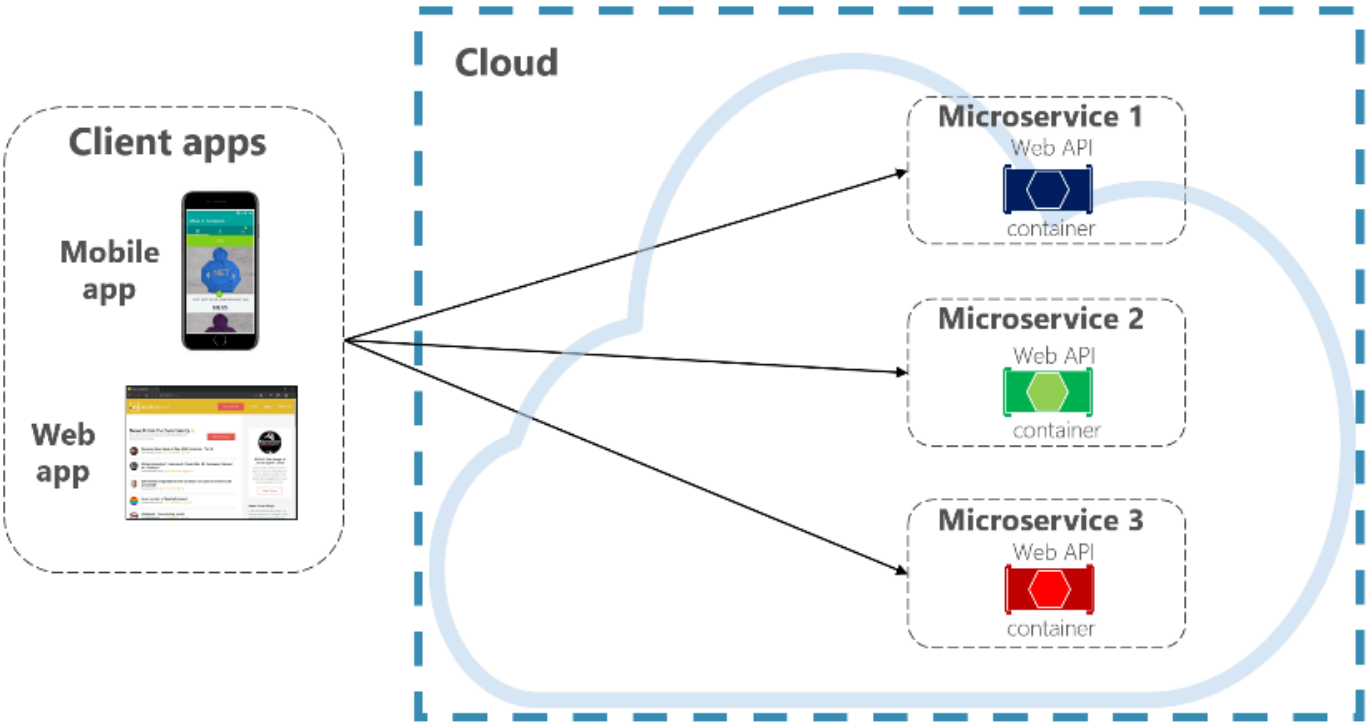
We'll explore the up and gRPC communication technology, and finally, we'll look new innovative communication patterns using service mesh technology.

We'll also see how the cloud providers such as Azure provides different kinds of backing services to support cloud-native communication.

Front-end client communication

In a cloud-native system, front-end clients (mobile, web, and desktop applications) require a communication channel to interact with independent backend microservices.

To keep things simple, a front-end client could directly communicate with the backend microservices.



With this approach, each microservice has a public endpoint that is accessible by front-end clients. In a production environment, you'd place a load balancer in front of the microservices, to route and distribute traffic proportionately.

Direct client to service communication

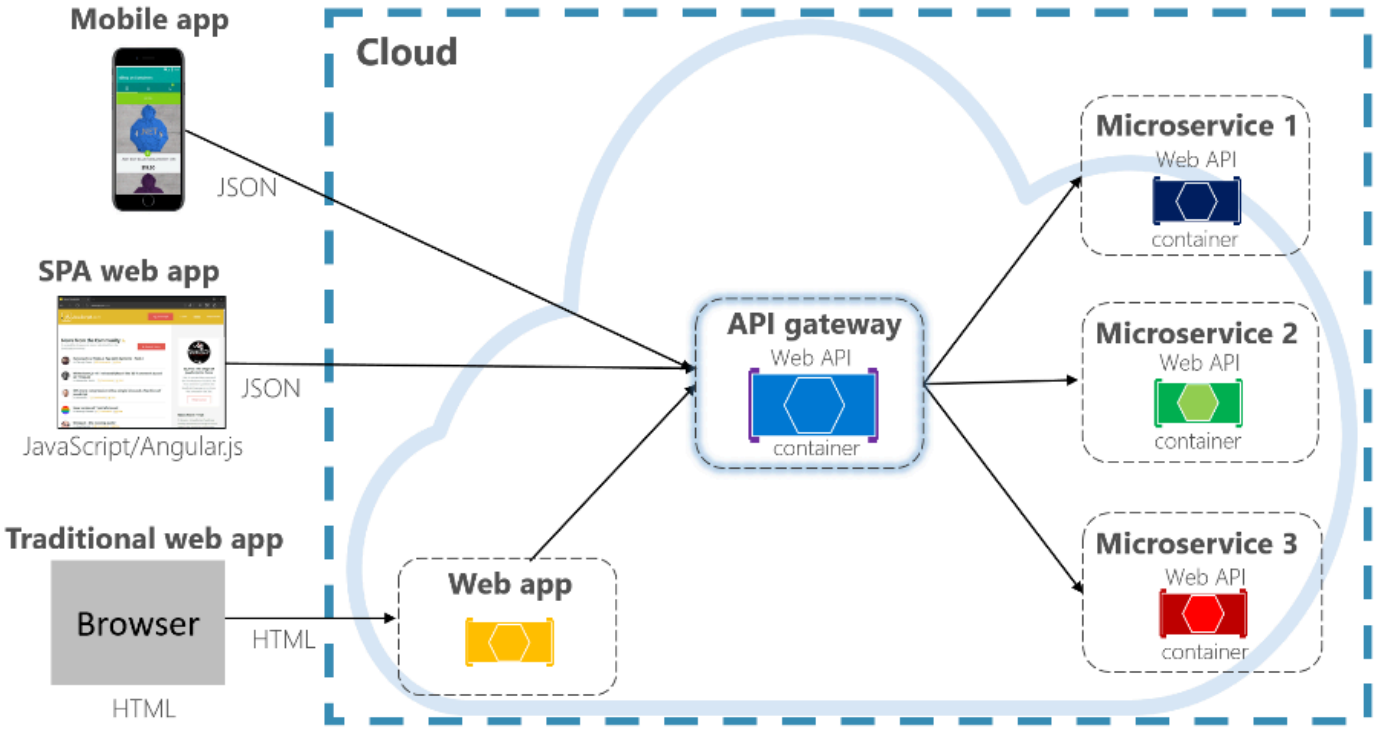
While simple to implement, direct client communication would be acceptable only for simple microservice applications. This pattern tightly couples front-end clients to core backend services, opening the door for many problems, including:

- Client susceptibility to backend service refactoring.
- A wider attack surface as core back-end services become directly exposed.
- Duplication of cross-cutting concerns across each microservice.

- Overly complex client code since clients must keep track of multiple endpoints and handle failures in a resilient way.

API gateway pattern

A widely accepted cloud design pattern is to implement an API Gateway Service between the front-end applications and back-end services.



In the previous figure, note how the API Gateway service abstracts the back-end core microservices. When implemented as a web API, it acts as a reverse proxy, routing incoming traffic to the internal microservices.

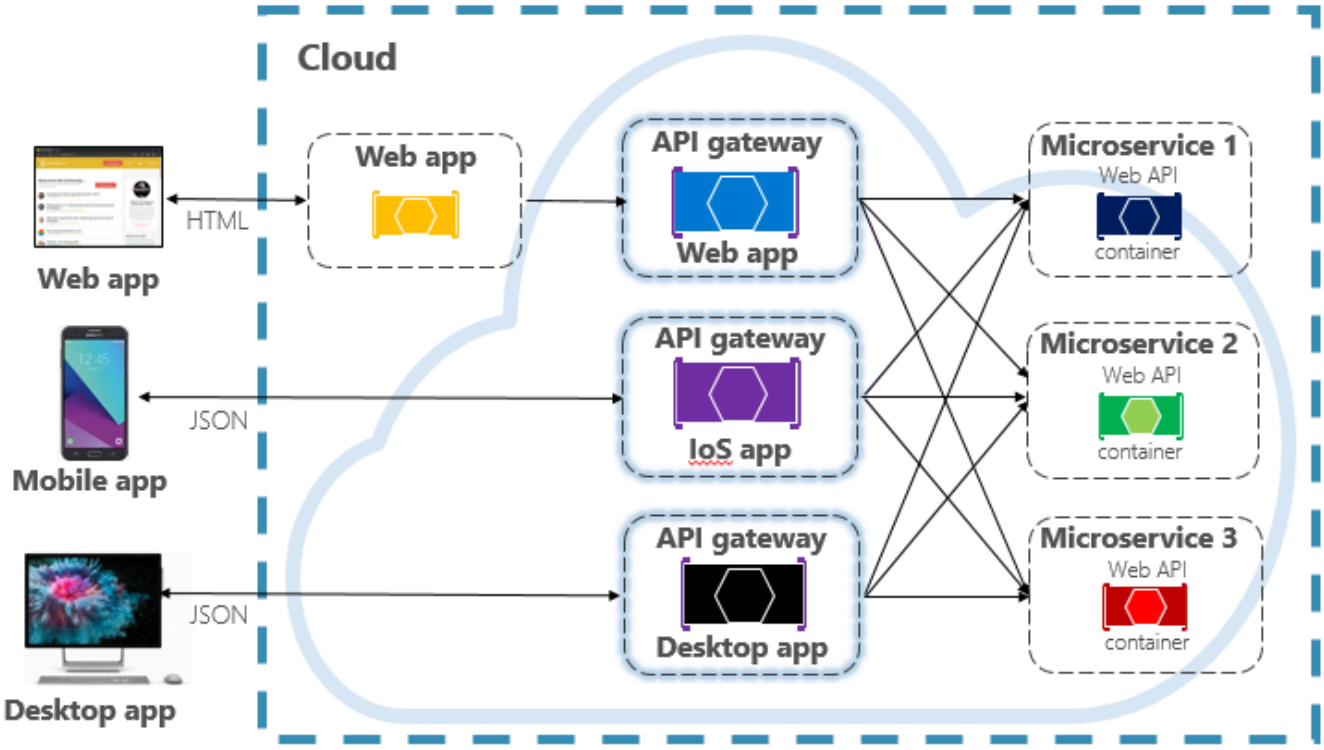
The gateway insulates the client from internal service partitioning and refactoring. If you change a back-end service, you accommodate for it in the gateway without breaking the client.

It's also your first line of defense for cross-cutting concerns, such as identity, caching, resiliency, metering, and throttling. Many cross-cutting concerns can be offloaded from the backend core services to the gateway, simplifying the backend services.

Care must be taken to keep the API Gateway simple and fast. Typically, business logic is kept out of the gateway. A complex gateway risks becoming a bottleneck and eventually a monolith itself.

Backend for frontend pattern

Larger systems often expose multiple API Gateways segmented by client type (mobile, web, desktop) or back-end functionality. This pattern provides direction for implementing multiple gateways.



Note in the previous figure how incoming traffic is sent to a specific API gateway, based upon the type of client used: web, mobile, or desktop app.

This approach makes sense as the capabilities of each device differ significantly across form factor, performance, and display limitations. Typically, mobile applications expose less functionality than a browser or desktop application. Each gateway can be optimized to match the capabilities and functionality of the corresponding device.

API Gateway service

When getting started with API Gateway services, you could build your own implementation. A quick search in GitHub will provide many examples.

For enterprise cloud-native applications however, there are several managed services from cloud providers that can help jump-start your efforts, including:

- Azure Application Gateway
- Amazon API Gateway

- Google API Gateway

This lesson will explain cloud-native communication patterns which are the ways in which applications and services communicate with each other in a cloud-native environment.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Recall the patterns used in cloud-native communication.

02 Define front-end communication.

03 Explain service mesh communication infrastructure.

Key Terms and Concepts

Key Term or Concept	Definition
Asynchronous	When communications are defined as asynchronous, it means that the sender of a communication does not “stay on hold” while the response comes back. The communication is sent, and the sender can then resume other activities. If a response is expected, the sender can establish an event-driven pattern to process the response when it comes in.

Key Term or Concept	Definition
Cross-cutting	Cross-cutting concerns are aspects of a program that affect several modules, without the possibility of being encapsulated in any of them. These concerns often cannot be cleanly decomposed from the rest of the system in both the design and implementation, and can result in either scattering (code duplication), tangling (significant dependencies between systems), or both.
Reverse Proxy	In computer networks, a reverse proxy is an application that sits in front of back-end applications and forwards client (e.g. browser) requests to those applications. Reverse proxies help increase scalability, performance, resilience and security. The resources returned to the client appear as if they originated from the web server itself.
Synchronous	Synchronous is an adjective that's defined as "occurring at the same time; coinciding in time; contemporaneous; simultaneous." In general, synchronous is used to describe things that happen at the same time. Synchronous communication involves real-time back-and-forth—talking face to face and video chatting are synchronous forms of communication.

Learning Activity



Consider the following questions related to cloud-native communication patterns and draft an answer for each:

1. What are the key differences between front-end client communication and service-to-service communication in cloud-native applications?

Click to see the answer.

2. Why is it important to consider communication patterns when designing a cloud-native application?

Click to see the answer.

3. How can gRPC improve service-to-service communication compared to REST?

Click to see the answer.

4. What are some benefits of using a service mesh for communication infrastructure?

Click to see the answer.

Real-time communication

Real-time, or push, communication is another option for front-end applications that communicate with back-end cloud-native systems over HTTP.

Applications, such as financial-tickers, online education, gaming, and job-progress updates, require instantaneous, real-time responses from the back end.

With normal HTTP communication, there's no way for the client to know when new data is available. The client must continually poll or send requests to the server. With real-time communication, the server can push new data to the client at any time.



Real-time communication concerns

Real-time systems are often characterized by high-frequency data flows and large numbers of concurrent client connections.

Manually implementing real-time connectivity can quickly become complex, requiring non-trivial infrastructure to ensure scalability and reliable messaging to connected clients. You could find yourself managing an instance of Redis Cache and a set of load balancers configured with sticky sessions for client affinity.

Real-time communication: Managed Services

Using a fully managed service simplifies real-time communication for your cloud-native applications. Technical implementation details like capacity provisioning, scaling, and persistent connections are abstracted away, and handled for you with a 99.9% service-level agreement. You can focus on application features, not infrastructure plumbing.

Examples of services include:

- Azure SignalR Service
- AWS AppSync
- Google Cloud Firestore

Service-to-service communication

Moving from the front-end client, we now address back-end microservices communicate with each other.

When constructing a cloud-native application, you'll want to be sensitive to how back-end services communicate with each other. Ideally, the less inter-service communication, the better. However, avoidance isn't always possible as back-end services often rely on one another to complete an operation.

There are several widely accepted approaches to implementing cross-service communication. The type of communication interaction will often determine the best approach.

Types of service-to-service communication

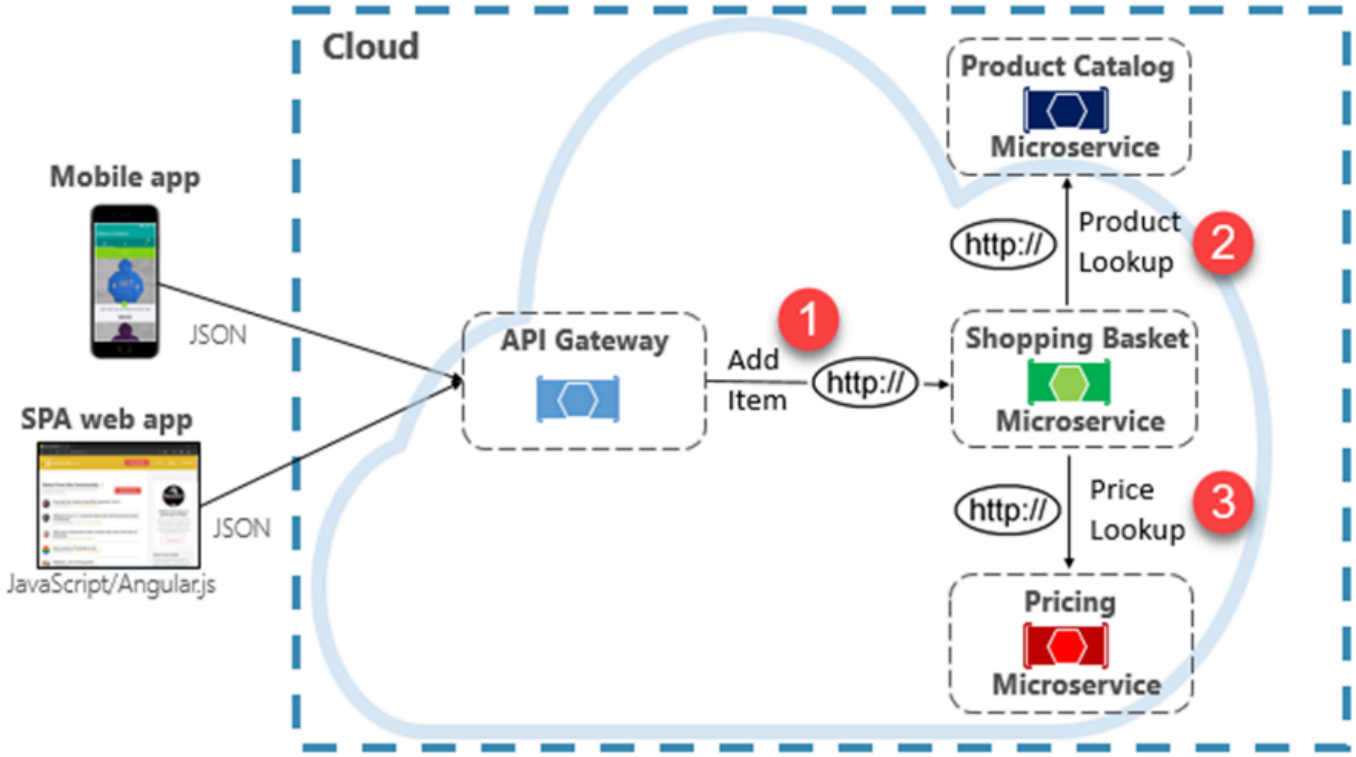
Here are types of communication calls used for service-to-service interaction:

- **Query** – when a calling microservice requires a response from a called microservice, such as, "Hey, give me the buyer information for a given customer Id."
- **Command** – when the calling microservice needs another microservice to execute an action but doesn't require a response, such as, "Hey, just ship this order."
- **Event** – when a microservice, called the publisher, raises an event that a state has changed, or an action has occurred. Other microservices, called subscribers, who are interested, can react to the event appropriately. The publisher and the subscribers aren't aware of each other.

Queries: Request/Response Messaging

Many times, one microservice might need to “query” another, requiring an immediate response to complete an operation. A shopping basket microservice may need product information and a price to add an item to its basket. There are many approaches for implementing query operations.

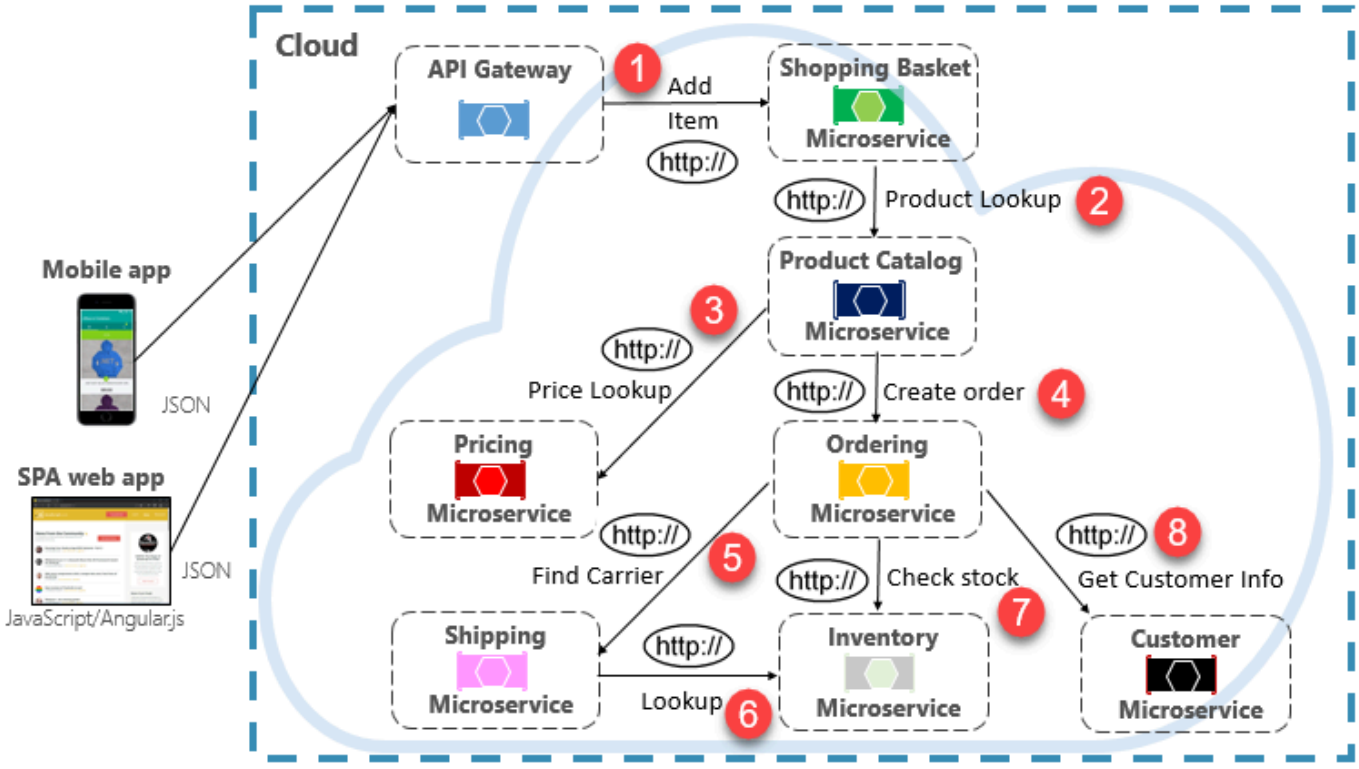
One option for implementing this scenario is for the calling back-end microservice to make direct HTTP requests to the microservices it needs to query.



Chaining HTTP queries

While direct HTTP calls between microservices are relatively simple to implement, care should be taken to minimize this practice. To start, these calls are always synchronous and will block the operation until a result is returned or the request times out. What were once self-contained, independent services, able to evolve independently and deploy frequently, now become coupled to each other. As coupling among microservices increase, their architectural benefits diminish.

Executing an infrequent request that makes a single direct HTTP call to another microservice might be acceptable for some systems. However, high-volume calls that invoke direct HTTP calls to multiple microservices aren't advisable. They can increase latency and negatively impact the performance, scalability, and availability of your system. Even worse, a long series of direct HTTP communication can lead to deep and complex chains of synchronous microservices calls.



You can certainly imagine the risk in the design shown in the previous image. What happens if Step #3 fails? Or Step #8 fails? How do you recover? What if Step #6 is slow because the underlying service is busy? How do you continue? Even if all works correctly, think of the latency this call would incur, which is the sum of the latency of each step.

The large degree of coupling in the previous image suggests the services weren't optimally modeled. It would behoove the team to revisit their design.

Materialized View pattern

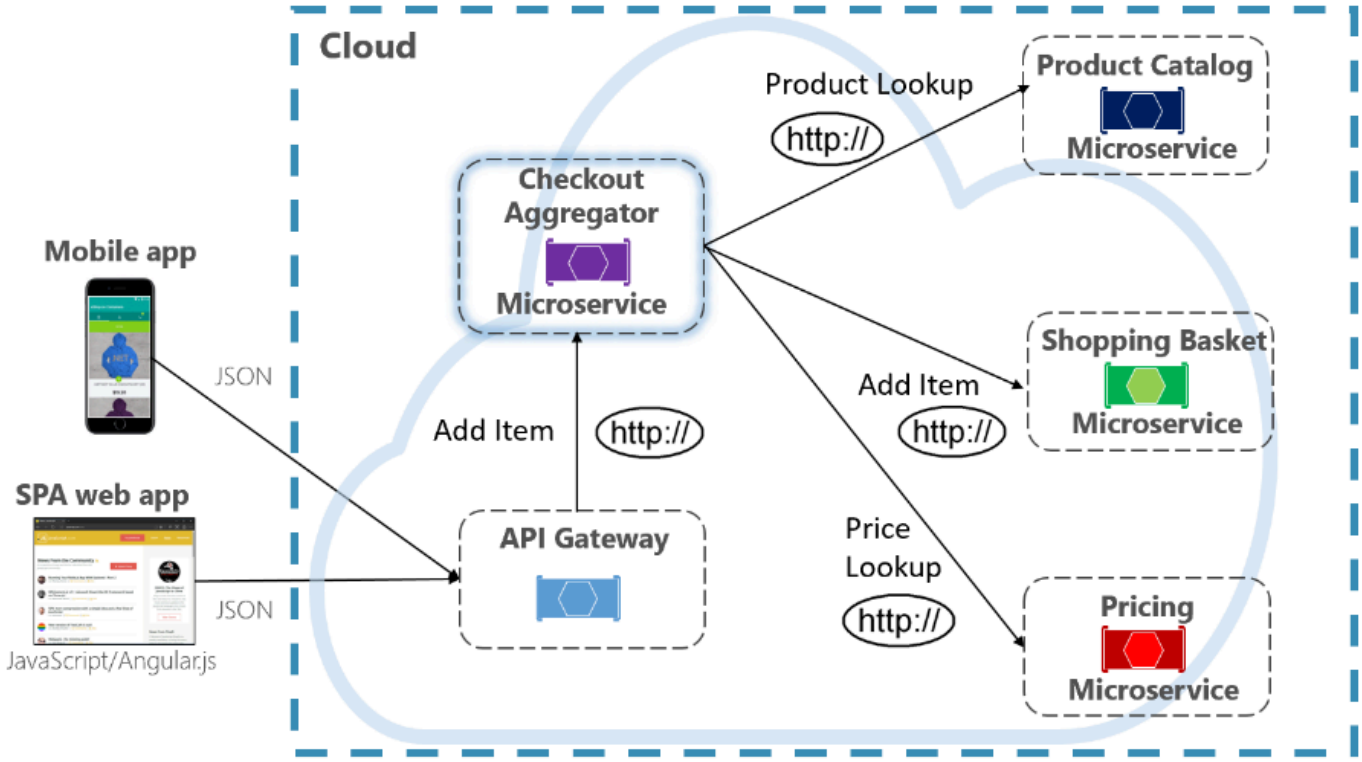
A popular option for removing microservice coupling is the Materialized View pattern. With this pattern, a microservice stores its own local, denormalized copy of data that's owned by other services.

Instead of the Shopping Basket microservice querying the Product Catalog and Pricing microservices, it maintains its own local copy of that data.

This pattern eliminates unnecessary coupling and improves reliability and response time. The entire operation executes inside a single process.

Service Aggregator Pattern

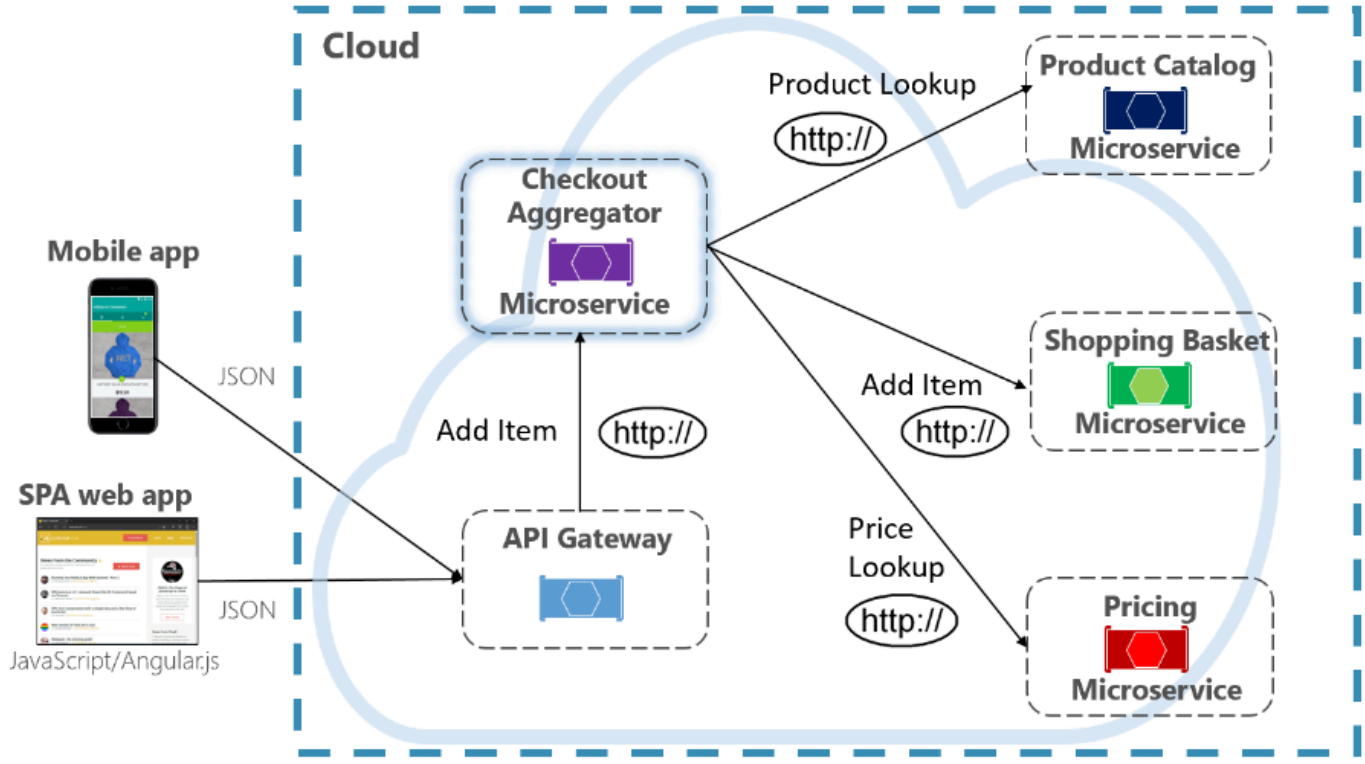
The Service Aggregator pattern isolates an operation that makes calls to multiple back-end microservices, centralizing its logic into a specialized microservice.



The purple checkout aggregator microservice in the previous figure orchestrates the workflow for the Checkout operation. It includes calls to several back-end microservices in a sequenced order. Data from the workflow is aggregated and returned to the caller. While it still implements direct HTTP calls, the aggregator microservice reduces direct dependencies among back-end microservices.

Request/Reply Pattern

Another approach for decoupling synchronous HTTP messages is a Request-Reply Pattern, which uses queuing communication. Communication using a queue is always a one-way channel, with a producer sending the message and consumer receiving it. With this pattern, both a request queue and response queue are implemented.



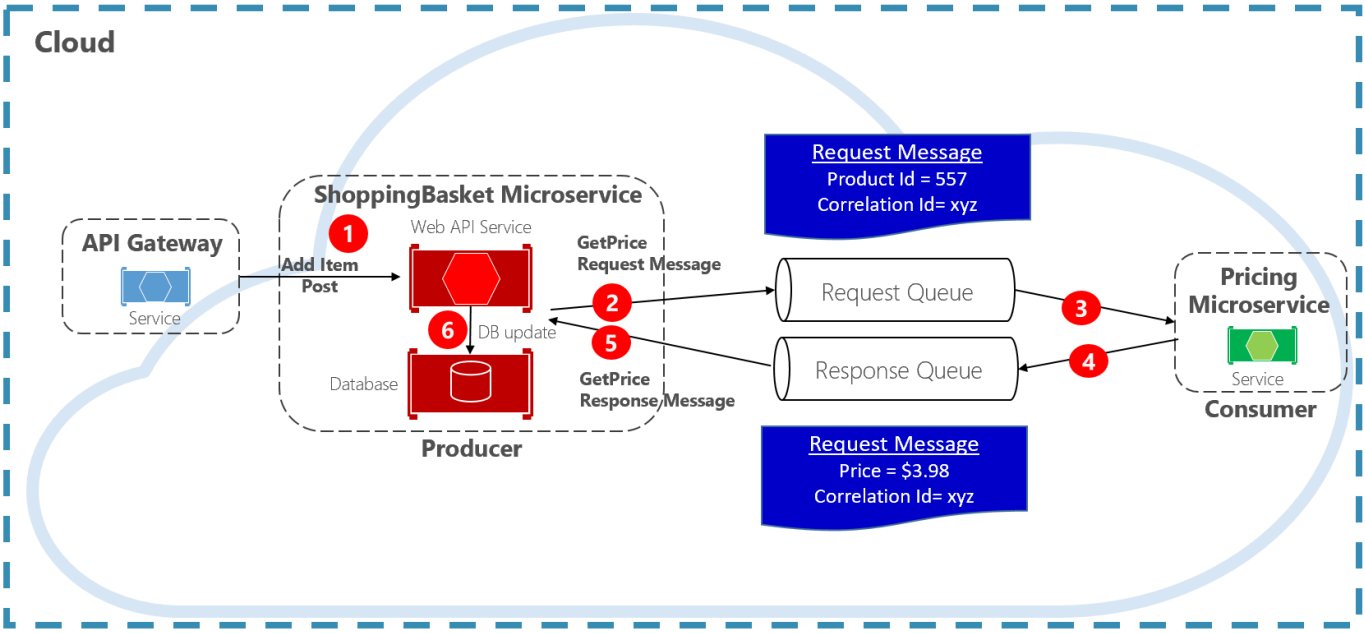
Here, the message producer creates a query-based message that contains a unique correlation ID and places it into a request queue. The consuming service dequeues the messages, processes it and places the response into the response queue with the same correlation ID. The producer service dequeues the message, matches it with the correlation ID and continues processing. We cover queues in detail in the next section.

The following resource provides a detailled explanation on the use of messaging patterns:

MESSAGING PATTERNSON (HTTPS://WWW.ENTERPRISEINTEGRATIONPATTERNS.COM/PATTERNS/MESSAGING/REQUESTREPLY.HTML)

Commands

Another type of communication interaction is a command. A microservice may need another microservice to perform an action. The Ordering microservice may need the Shipping microservice to create a shipment for an approved order. In the following figure, one microservice, called a Producer, sends a message to another microservice, the Consumer, commanding it to do something.



Most often, the Producer doesn't require a response and can “fire-and-forget” the message. If a reply is needed, the Consumer sends a separate message back to Producer on another channel. A command message is best sent asynchronously with a message queue, supported by a lightweight message broker. In the previous diagram, note how a queue separates and decouples both services.

A message queue is an intermediary construct through which a producer and consumer pass a message. Queues implement an asynchronous, point-to-point messaging pattern. The Producer knows where a command needs to be sent and routes appropriately. The queue guarantees that a message is processed by exactly one of the consumer instances that are reading from the channel. In this scenario, either the producer or consumer service can scale out without affecting the other. As well, technologies can be disparate on each side, meaning that we might have a Java microservice calling a Golang microservice, for example.

In previous lessons, we talked about backing services. Backing services are ancillary resources upon which cloud-native systems depend. Message queues are backing services.

The Azure cloud supports two types of message queues that your cloud-native systems can consume to implement command messaging:

- Azure Storage Queues
- Azure Service Bus Queues.

Events

Message queuing is an effective way to implement communication where a producer can asynchronously send a consumer a message. However, what happens when many different consumers are interested in the same message? A dedicated message queue for each consumer wouldn't scale well and would become difficult to manage.

To address this scenario, we move to the third type of message interaction, the event. One microservice announces that an action has occurred. Other microservices, if interested, react to the action, or event. This is also known as the event-driven architectural style.

The following Microsoft resource provides a detailed explanation on the use of the event-driven architecture style:

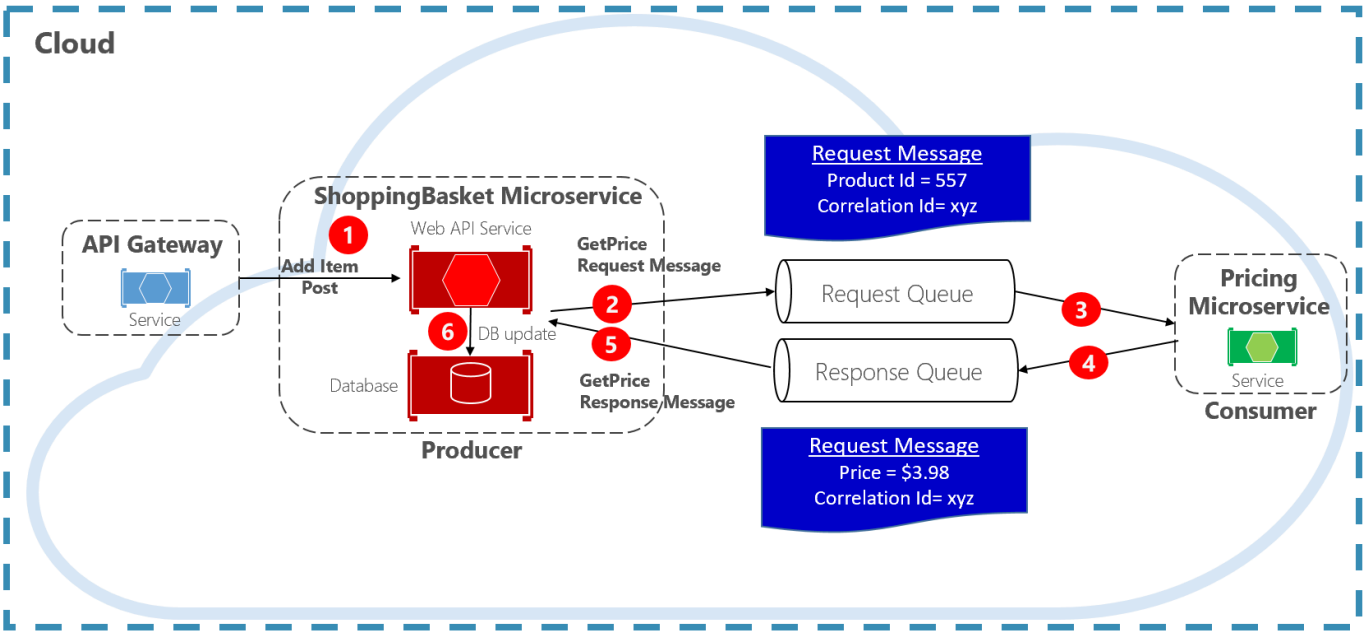
EVENT-DRIVEN ARCHITECTURE STYLE (HTTPS://LEARN.MICROSOFT.COM/EN-US/AZURE/ARCHITECTURE/GUIDE/ARCHITECTURE-STYLES/EVENT-DRIVEN)

Eventing is a two-step process. For a given state change, a microservice publishes an event to a message broker, making it available to any other interested microservice. The interested microservice is notified by subscribing to the event in the message broker. You use the Publish/Subscribe pattern to implement event-based communication.

The following Microsoft resource provides a detailed explanation on the use of the Publisher-Subscriber pattern: **(https://learn.microsoft.com/en-us/azure/architecture/patterns/publisher-subscriber)**

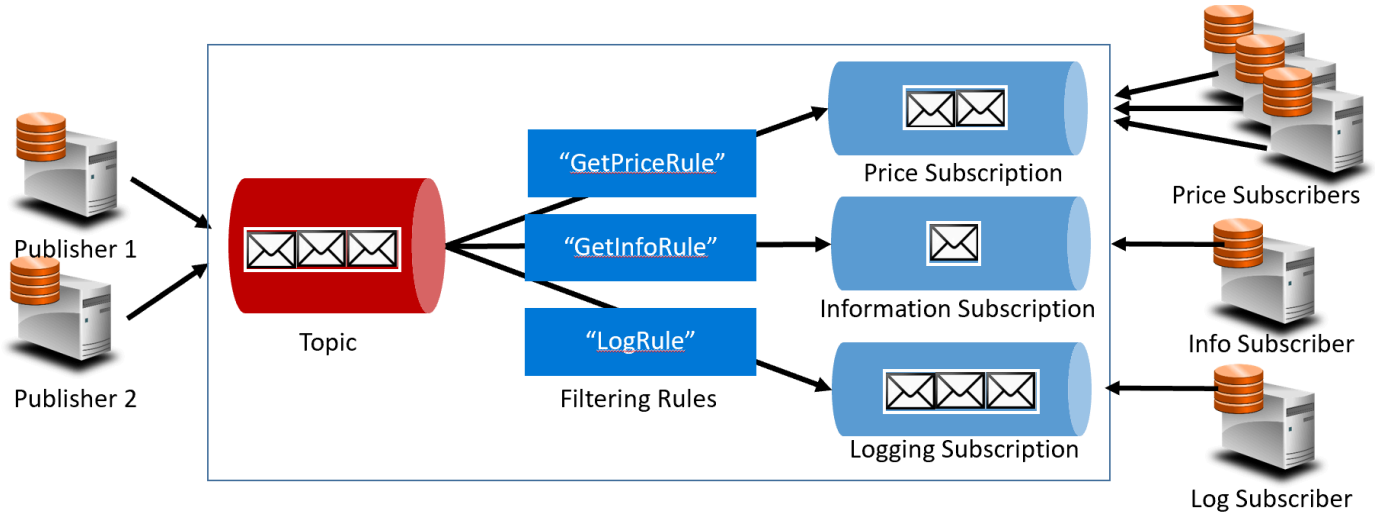
PUBLISHER-SUBSCRIBER PATTERN (../../DAYLIGHT_HTML_TEMPLATE/MODULE_TEMPLATES/?OU=890975)

The following figure shows a shopping basket microservice publishing an event with two other microservices subscribing to it. Note the event bus component that sits in the middle of the communication channel. It's a custom class that encapsulates the message broker and decouples it from the underlying application. The ordering and inventory microservices independently operate the event with no knowledge of each other, nor the shopping basket microservice. When the registered event is published to the event bus, they act upon it.



Topic architecture

With eventing, we move from queuing technology to topics. A topic is similar to a queue but supports a one-to-many messaging pattern. One microservice publishes a message and multiple subscribing microservices can choose to receive and act upon that message.



In the previous figure, publishers send messages to the topic. At the end, subscribers receive messages from subscriptions. In the middle, the topic forwards messages to subscriptions based on a set of rules, shown in dark blue boxes. Rules act as a filter that forward specific messages to a subscription. Here, a "GetPrice" event would be sent to the price and logging subscriptions as the logging subscription has chosen to receive all messages. A "GetInformation" event would be sent to the information and logging subscriptions.

---

gRPC

gRPC is a modern, high-performance framework that evolves the age-old remote procedure call (RPC) protocol.

At the application level, gRPC streamlines messaging between clients and back-end services. Originating from Google, gRPC is open source and part of the Cloud Native Computing Foundation (CNCF) ecosystem of cloud-native offerings.

CNCF considers gRPC an incubating project. Incubating means end users are using the technology in production applications, and the project has a healthy number of contributors.



Service Mesh communication infrastructure

Throughout this lesson, we've explored the challenges of microservice communication.

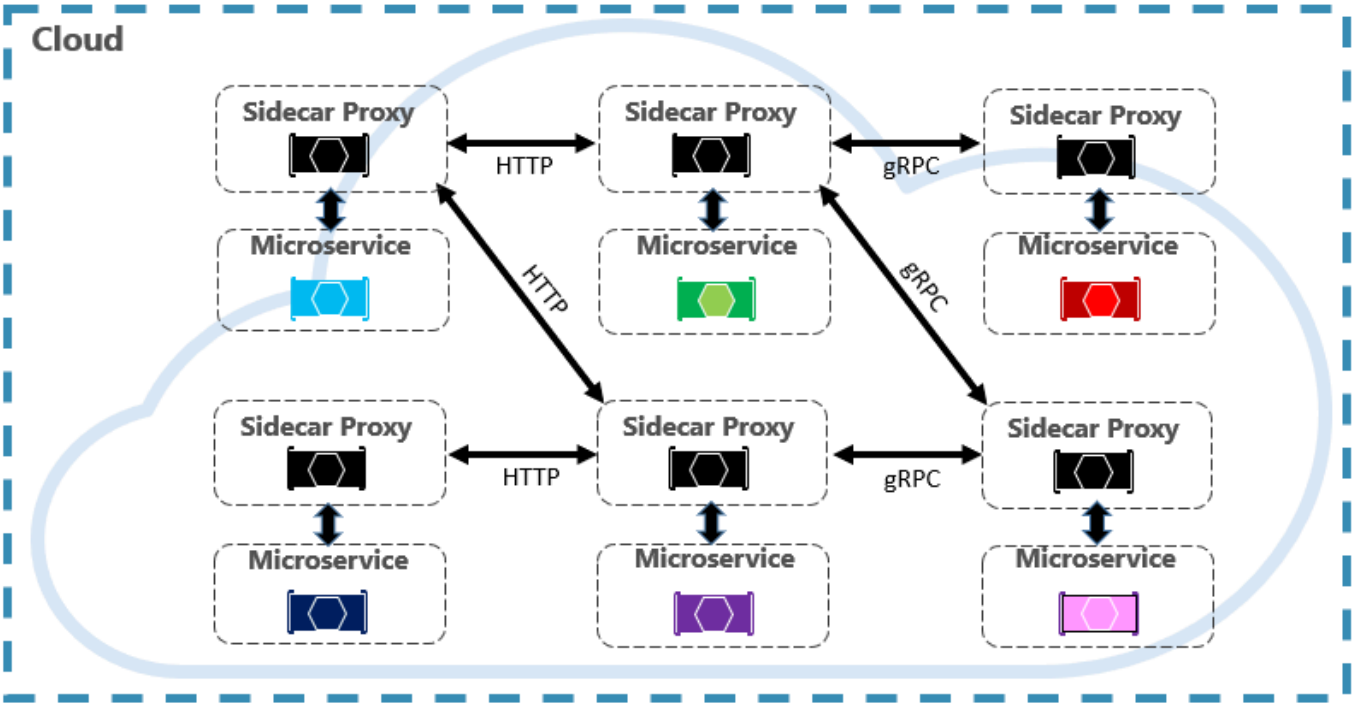
We explored different approaches for implementing synchronous HTTP communication and asynchronous messaging. In each of the cases, the developer is burdened with implementing communication code. Communication code is complex and time intensive. Incorrect decisions can lead to significant performance issues.

A more modern approach to microservice communication centers around a new and rapidly evolving technology entitled Service Mesh. A service mesh is a configurable infrastructure layer with built-in capabilities to handle service-to-service communication, resiliency, and many cross-cutting concerns. It moves the responsibility for these concerns out of the microservices and into service mesh layer. Communication is abstracted away from your microservices.

Service mesh has gained in popularity and have become a popular implementation choice for cloud-native apps. Read more about the technology in the following blog: **(https://www.nginx.com/blog/what-is-a-service-mesh/)**

WHAT IS A SERVICE MESH? (HTTPS://WWW.NGINX.COM/BLOG/WHAT-IS-A-SERVICE-MESH/)

A key component of a service mesh is a proxy. In a cloud-native application, an instance of a proxy is typically co-located with each microservice. While they execute in separate processes, the two are closely linked and share the same lifecycle. This pattern is known as the Sidecar pattern.



Note in the previous figure how messages are intercepted by a proxy that runs alongside each microservice. Each proxy can be configured with traffic rules specific to the microservice. It understands messages and can route them across your services and the outside world.

Along with managing service-to-service communication, the Service Mesh provides support for service discovery and load balancing. Once configured, a service mesh is highly functional. The mesh retrieves a corresponding pool of instances from a service discovery endpoint. It sends a request to a specific service instance, recording the latency and response type of the result. It chooses the instance most likely to return a fast response based on different factors, including the observed latency for recent requests.

A service mesh manages traffic, communication, and networking concerns at the application level. It understands messages and requests. A service mesh typically integrates with a container orchestrator. Kubernetes supports an extensible architecture in which a service mesh can be added.

Caching in a cloud-native app

The benefits of caching are well understood. The technique works by temporarily copying frequently accessed data from a backend data store to fast storage that is located closer to the application. Caching is often implemented where:

- Data remains relatively static.
- Data access is slow, especially compared to the speed of the cache.
- Data is subject to high levels of contention.

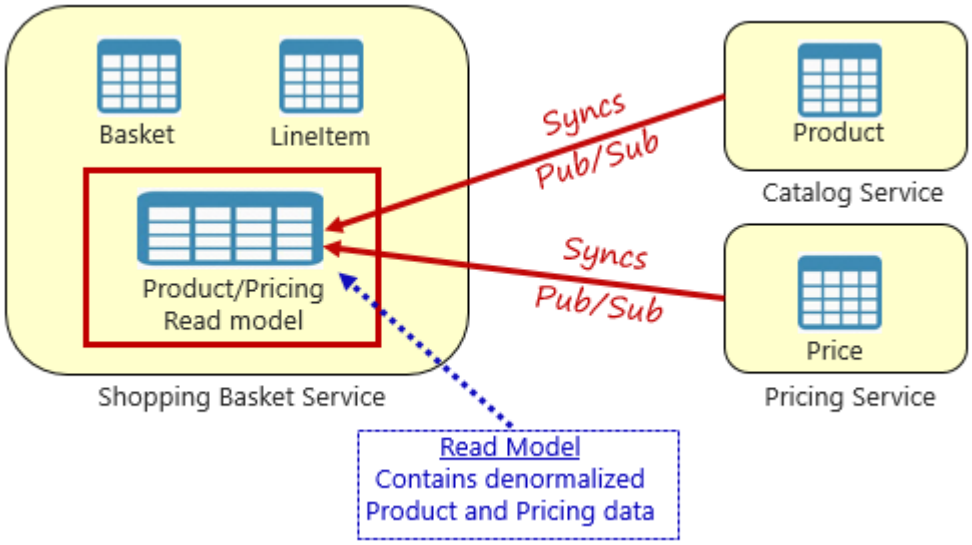
Caching can increase performance, scalability, and availability for individual microservices and the system as a whole. It reduces the latency and contention of handling large volumes of concurrent requests to a data store. As data volume and the number of users increase, the greater the benefits of caching become.

Caching is most effective when a client repeatedly reads data that is immutable or that changes infrequently. Examples include reference information such as product and pricing information, or shared static resources that are costly to construct.

While microservices should be stateless, a distributed cache can support concurrent access to session state data when absolutely required. Also consider caching to avoid repetitive computations. If an operation transforms data or performs a complicated calculation, cache the result for subsequent requests.

Caching architecture

Cloud native applications typically implement a distributed caching architecture. The cache is hosted as a cloud-based backing service, separate from the microservices.



In the previous figure, note how the cache is independent of and shared by the microservices. In this scenario, the cache is invoked by the API Gateway. The gateway serves as a front end for all incoming requests. The distributed cache increases system responsiveness by returning cached data whenever possible. Additionally, separating the cache from the services allows the cache to scale up or out independently to meet increased traffic demands.

The previous figure presents a common caching pattern known as the cache-aside pattern. For an incoming request, you first query the cache (step #1) for a response. If found, the data is returned immediately. If the data doesn't exist in the cache (known as a cache miss), it's retrieved from a local database in a downstream service (step #2). It's then written to the cache for future requests (step #3) and returned to the caller. Care must be taken to periodically evict cached data so that the system remains timely and consistent.

As a shared cache grows, it might prove beneficial to partition its data across multiple nodes. Doing so can help minimize contention and improve scalability. Many caching services support the ability to dynamically add and remove nodes and rebalance data across partitions. This approach typically involves clustering. Clustering exposes a collection of federated nodes as a seamless, single cache. Internally, however, the data is dispersed across the nodes following a predefined distribution strategy that balances the load evenly.

Elasticsearch in a cloud-native app

Elasticsearch is a distributed search and analytics system that enables complex search capabilities across diverse types of data. It's open source and widely popular. Consider how the following companies integrate Elasticsearch into their application:

- It is used by Wikipedia for full-text and incremental (search as you type) searching.
- It is used by GitHub to index and expose over 8 million code repositories.
- It is used by Docker to make its container library discoverable.



Elasticsearch is built on top of the Apache Lucene full-text search engine. Lucene provides high-performance document indexing and querying. It indexes data with an inverted indexing scheme – instead of mapping pages to keywords, it maps keywords to pages just like a glossary at the end of a book.

Lucene has powerful query syntax capabilities and can query data by:

- Term (a full word)
- Prefix (starts-with word)
- Wildcard (using "*" or "?" filters)
- Phrase (a sequence of text in a document)
- Boolean value (complex searches combining queries)

While Lucene provides low-level plumbing for searching, Elasticsearch provides the server that sits on top of Lucene. Elasticsearch adds higher-level functionality to simplify working Lucene, including a RESTful API to access Lucene's indexing and searching functionality. It also provides a distributed infrastructure capable of massive scalability, fault tolerance, and high availability.

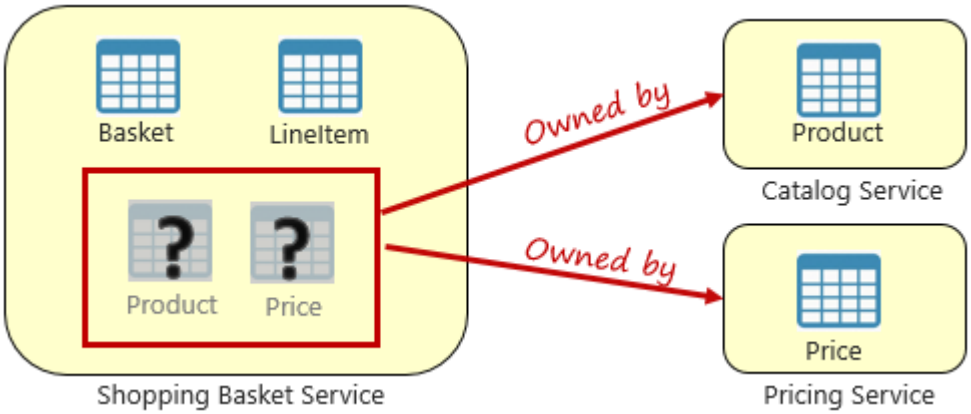
Challenges of cloud-native data patterns

While encapsulating data into separate microservices can increase agility, performance, and scalability, it also presents many challenges.

We discuss these challenges along with patterns and practices to help overcome them.

Challenge: Cross-service queries

While microservices are independent and focus on specific functional capabilities, like inventory, shipping, or ordering, they frequently require integration with other microservices. Often, the integration involves one microservice querying another for data.



In the preceding figure, we see a shopping basket microservice that adds an item to a user's shopping basket. While the data store for this microservice contains basket and line-item data, it doesn't maintain product or pricing data. Instead, those data items are owned by the catalog and pricing microservices. This aspect presents a problem. How can the shopping basket microservice add a product to the user's shopping basket when it doesn't have product nor pricing data in its database?

One option covered in previous lessons is a direct HTTP call from the shopping basket to the catalog and pricing microservices. However, you will recall that synchronous HTTP calls couple microservices together, reducing their autonomy and diminishing their architectural benefits.

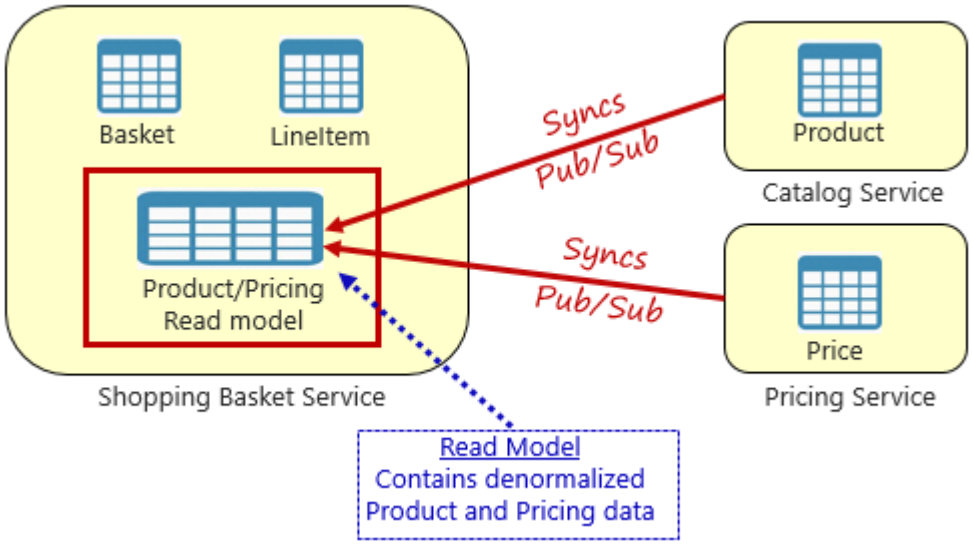
We could also implement a request-reply pattern with separate inbound and outbound queues for each service. However, this pattern is complicated and requires plumbing to correlate request and response messages. While it does decouple the backend microservice calls, the calling service must still synchronously wait for the call to complete. Network congestion, transient faults, or an overloaded microservice and can result in long running and even failed operations

---

Solution: Materialized View Pattern

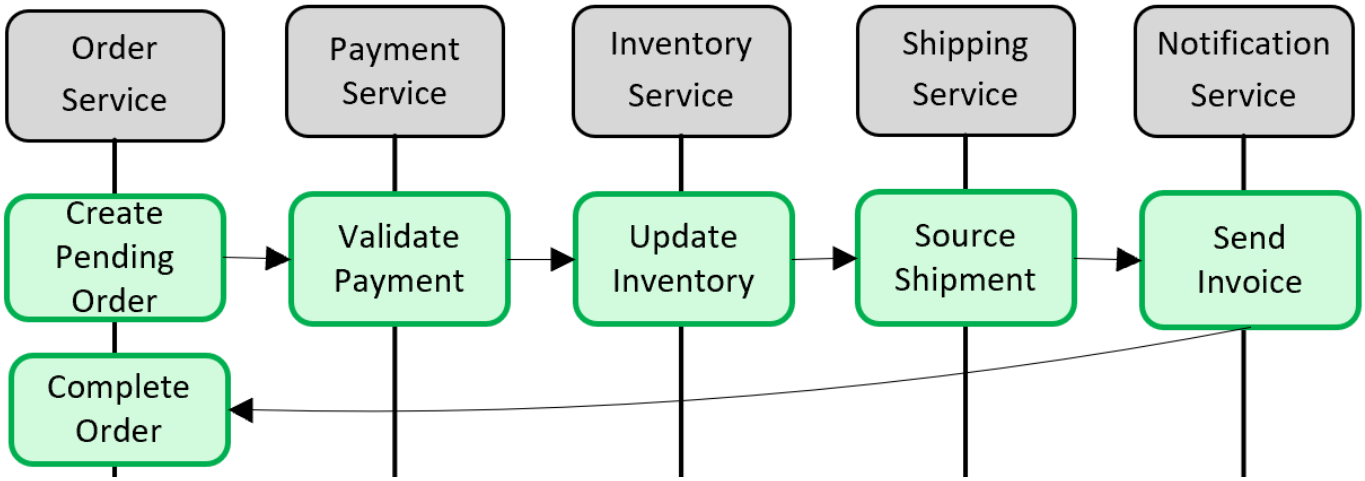
A widely accepted pattern for removing cross-service dependencies is the Materialized View Pattern. With this pattern, you place a local data table (known as a read model) in the shopping basket service. This table contains a denormalized copy of the data needed from the product and pricing microservices. Copying the data directly into the shopping basket microservice eliminates the need for expensive cross-service calls. With the data local to the service, you improve the service's response time and reliability. Additionally, having its own copy of the data makes the shopping basket service more resilient. If the catalog service should become unavailable, it wouldn't directly impact the shopping basket service. The shopping basket can continue operating with the data from its own store.

The catch with this approach is that you now have duplicate data in your system. However, strategically duplicating data in cloud-native systems is an established practice and not considered an anti-pattern, or bad practice. Keep in mind that one and only one service can own a dataset and have authority over it. You'll need to synchronize the read models when the system of record is updated. Synchronization is typically implemented via asynchronous messaging with a publish/subscribe pattern.



Challenge: Distributed transactions

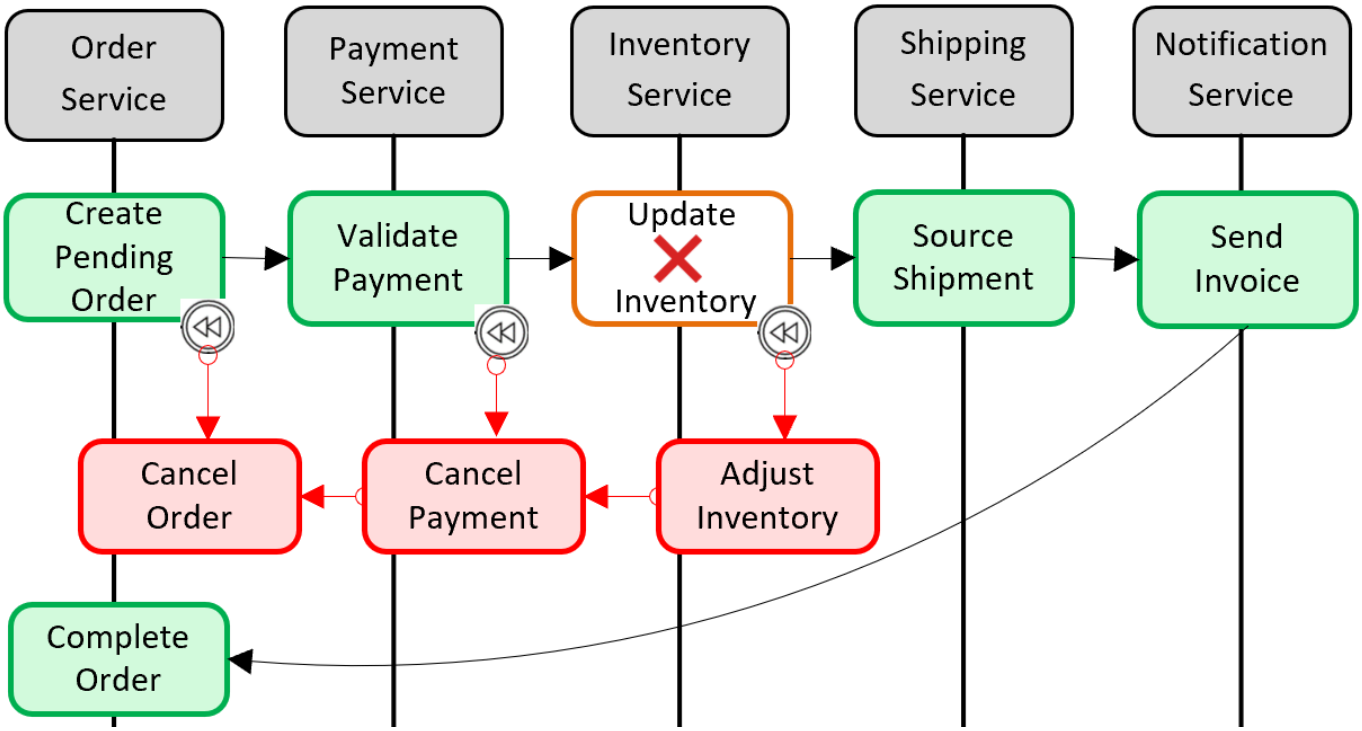
While querying data across microservices is difficult, implementing a transaction across several microservices is even more complex. The inherent challenge of maintaining data consistency across independent data sources in different microservices can't be understated. The lack of distributed transactions in cloud-native applications means that you must manage distributed transactions programmatically. You move from a world of immediate consistency to that of eventual consistency.



In the preceding figure, five independent microservices participate in a distributed transaction that creates an order. Each microservice maintains its own data store and implements a local transaction for its store. To create the order, the local transaction for each individual microservice must succeed, or all must abort and roll back the operation. While built-in transactional support is available inside each of the microservices, there's no support for a distributed transaction that would span across all five services to keep data consistent. Instead, you must construct this distributed transaction programmatically.

Solution: Saga pattern

A popular pattern for adding distributed transactional support is the Saga pattern. It's implemented by grouping local transactions together programmatically and sequentially invoking each one. If any of the local transactions fail, the Saga aborts the operation and invokes a set of compensating transactions. The compensating transactions undo the changes made by the preceding local transactions and restore data consistency. Observe the following figure which shows a failed transaction with the Saga pattern.



In the previous figure, the “Update Inventory” operation has failed in the Inventory microservice. The Saga invokes a set of compensating transactions (in red) to adjust the inventory counts, cancel the payment and the order, and return the data for each microservice back to a consistent state.

Saga patterns are typically choreographed as a series of related events or orchestrated as a set of related commands.

Consult the following Microsoft resource to learn more about the Saga Pattern for cloud-native.

COMPENSATING TRANSACTION PATTERN (HTTPS://LEARN.MICROSOFT.COM/EN-US/AZURE/ARCHITECTURE/PATTERNS/COMPENSATING-TRANSACTION)

Challenge: High volume data

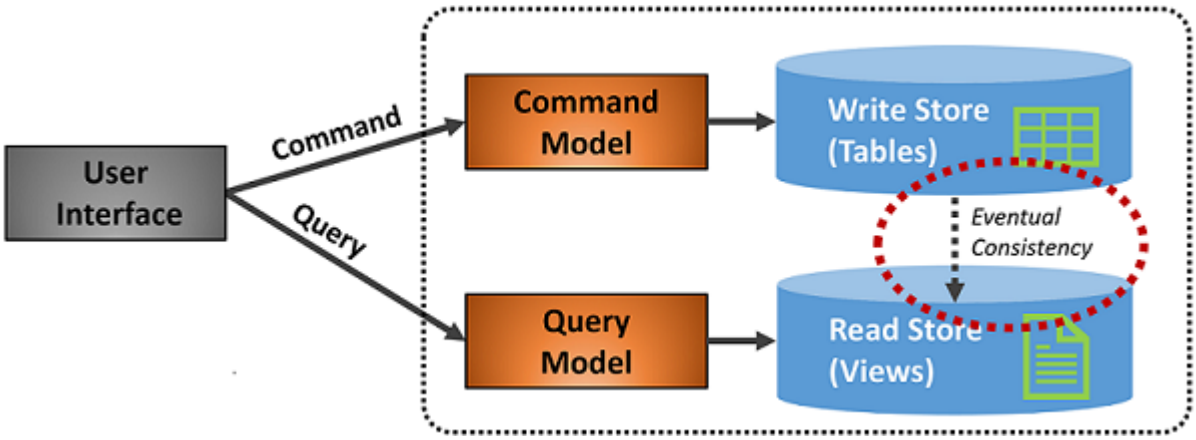
Large cloud-native applications often support high-volume data requirements. In these scenarios, traditional data storage techniques can cause bottlenecks.



For complex systems that deploy on a large scale, both Command and Query Responsibility Segregation (CQRS) and Event Sourcing may improve application performance.

Solution: CQRS

For normal scenarios, the same entity model and data repository object are used for both read and write operations. However, a high-volume data scenario can benefit from separate models and data tables for reads and writes. To improve performance, the read operation could query against a highly denormalized representation of the data to avoid expensive repetitive table joins and table locks. The write operation, known as a command, would update against a fully normalized representation of the data that would guarantee consistency. You then need to implement a mechanism to keep both representations in sync. Typically, whenever the write table is modified, it publishes an event that replicates the modification to the read table.



In the previous figure, separate command and query models are implemented. Each data write operation is saved to the write store and then propagated to the read store. Pay close attention to how the data propagation process operates on the principle of eventual consistency. The read model eventually synchronizes with the write model, but there may be some lag in the process.

This separation enables reads and writes to scale independently. Read operations use a schema optimized for queries, while the writes use a schema optimized for updates. Read queries go against denormalized data, while complex business logic can be applied to the write model. As well, you might impose tighter security on write operations than those exposing reads.

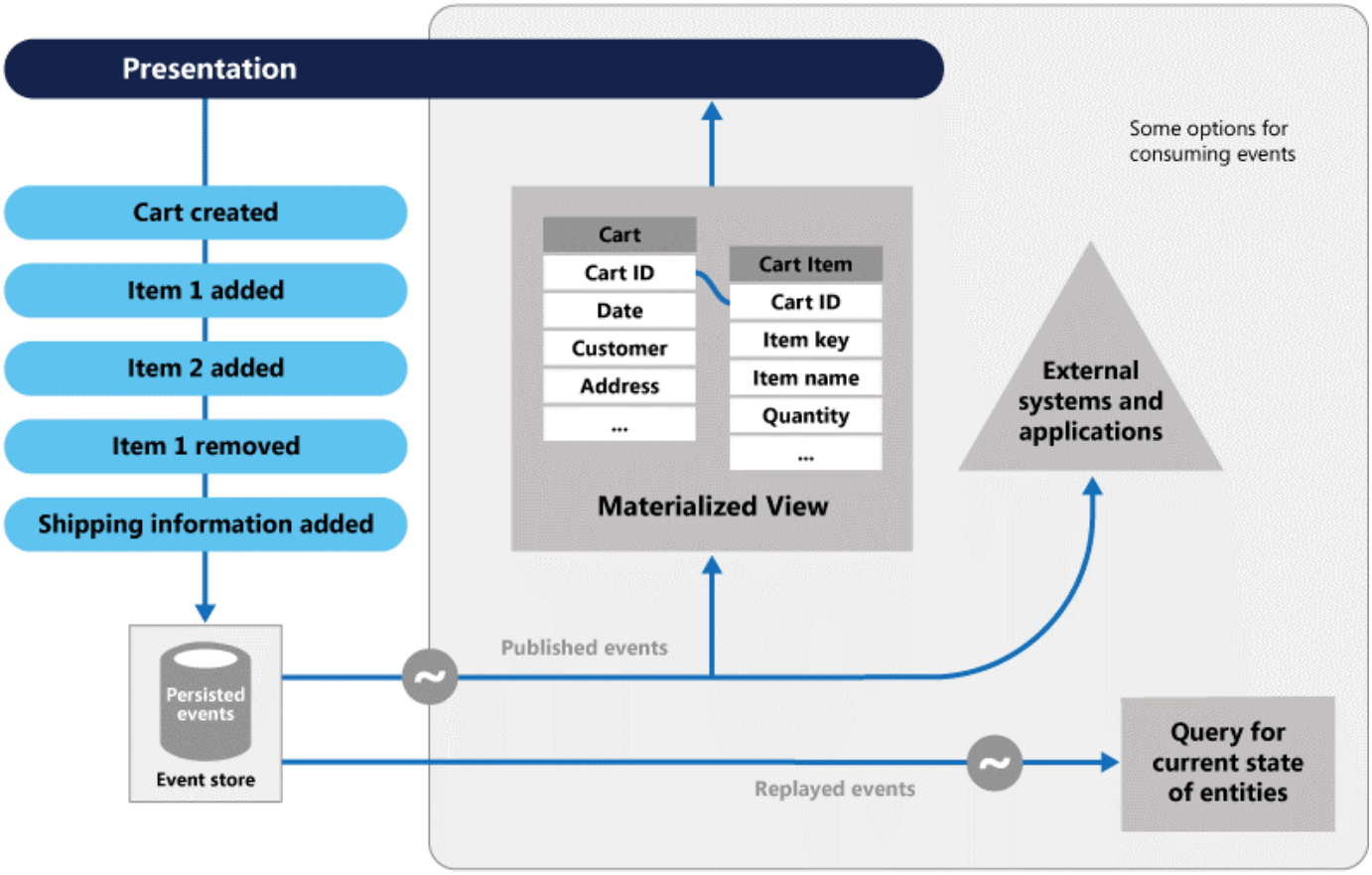
Implementing CQRS can improve application performance for cloud-native services. However, it does result in a more complex design. Apply this principle carefully and strategically to those sections of your cloud-native application that will benefit from it.

Solution: Event sourcing

Another approach to optimizing high volume data scenarios involves Event Sourcing.

A system typically stores the current state of a data entity. If a user changes their phone number, for example, the customer record is updated with the new number. We always know the current state of a data entity, but each update overwrites the previous state. In most cases, this model works fine. In high volume systems, however, overhead from transactional locking and frequent update operations can impact database performance, responsiveness, and limit scalability.

Event Sourcing takes a different approach to capturing data. Each operation that affects data is persisted to an event store. Instead of updating the state of a data record, we append each change to a sequential list of past events - similar to an accountant's ledger. The Event Store becomes the system of record for the data. It is used to propagate various materialized views within the bounded context of a microservice.



In the previous figure, note how each entry (in blue) for a user's shopping cart is appended to an underlying event store. In the adjoining materialized view, the system projects the current state by replaying all the events associated with each shopping cart. This view, or read model, is then exposed back to the UI. Events can also be integrated with external systems and applications or queried to determine the current state of an entity. With this approach, you maintain history. You know not only the current state of an entity, but also how you reached this state.

Mechanically speaking, event sourcing simplifies the write model. There are no updates or deletes. Appending each data entry as an immutable event minimizes contention, locking, and concurrency conflicts associated with relational databases. Building read models with the materialized view pattern enables you to decouple the view from the write model and choose the best data store to optimize the needs of your application UI.
For this pattern, there are data store models that directly supports event sourcing. Azure Cosmos DB, MongoDB, Cassandra, CouchDB, and RavenDB are good candidates.

As with all patterns and technologies, implement strategically and when needed. While event sourcing can provide increased performance and scalability, it comes at the expense of complexity and a learning curve.

Cloud-native data patterns

As we've seen throughout this course, a cloud-native approach changes the way you design, deploy, and manage applications. It also changes the way you manage and store data.

In this lesson, we dive deeper into the use of data sources in cloud-native application development.

Monolithic vs. Cloud-native Application

In traditional development, i.e., using monolithic applications, business service components collocate together in a shared services tier, sharing data from a single relational database.

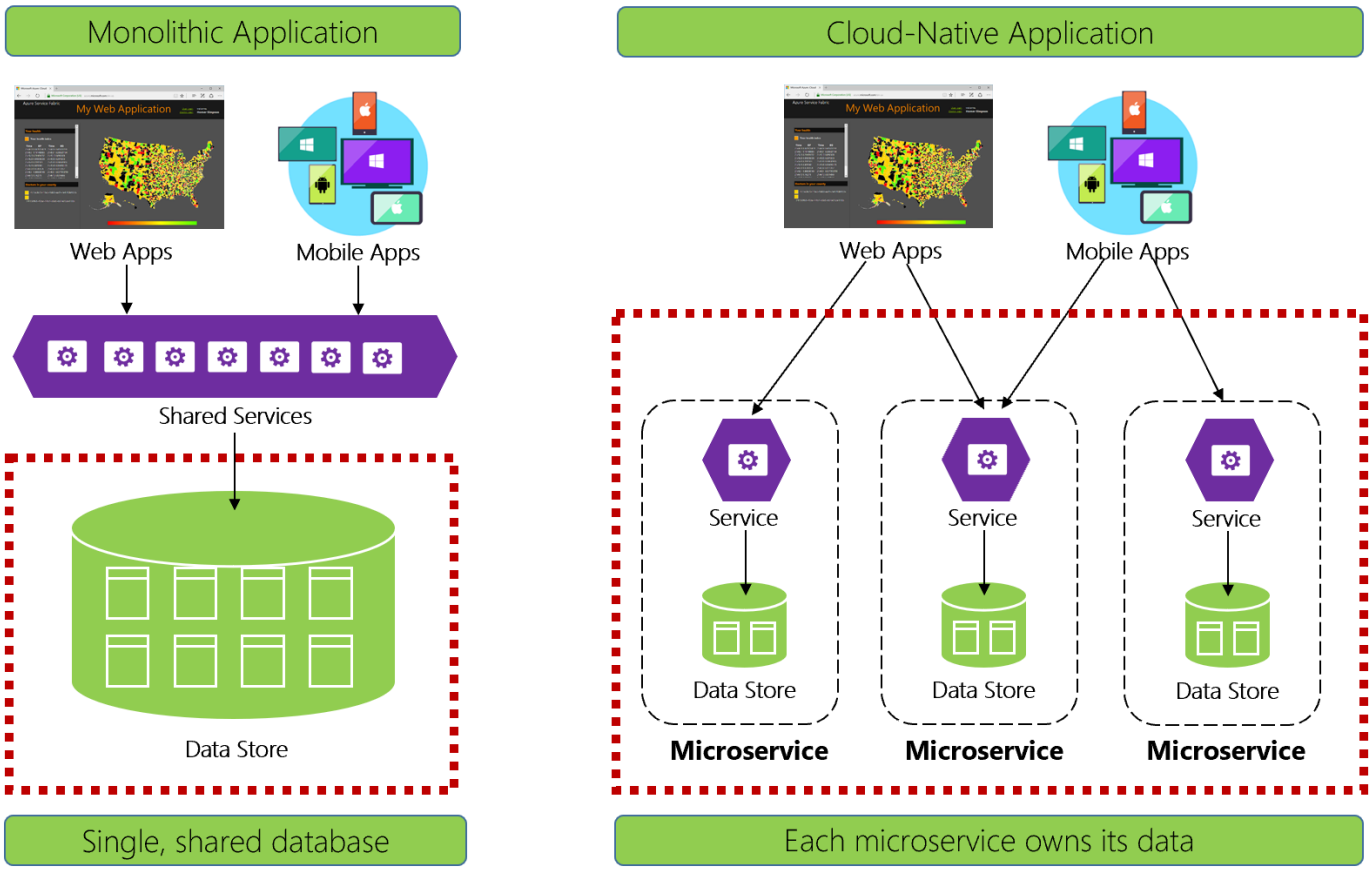


Figure 5-1

In many ways, a single database keeps data management simple. Querying data across multiple tables is straightforward. Changes to data update together or they all rollback. ACID transactions guarantee strong and immediate consistency.

Designing for cloud-native, we take a different approach.

On the right-side of Figure 5-1, note how business functionality segregates into small, independent microservices. Each microservice encapsulates a specific business capability and its own data. The monolithic database decomposes into a distributed data model with many smaller databases, each aligning with a microservice. When the smoke clears, we emerge with a design that exposes a database per microservice.

Why use a database-per-microservice approach

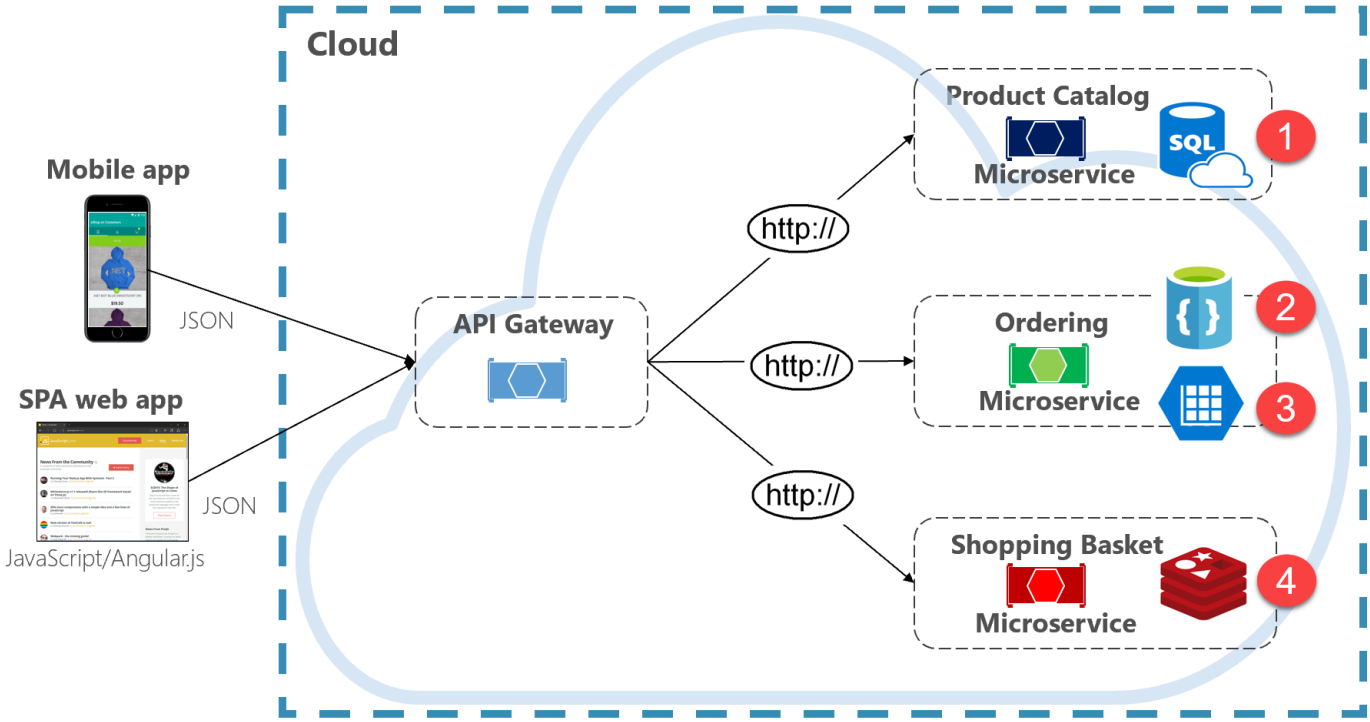
This database per microservice provides many benefits, especially for systems that must evolve rapidly and support massive scale. With this model:

- Domain data is encapsulated within the service.
- Data schema can evolve without directly impacting other services.
- Each data store can independently scale.
- A data store failure in one service won't directly impact other services.

Polyglot data persistence

Segregating data also enables each microservice to implement the data store type that is best optimized for its workload, storage needs, and read/write patterns. Choices include relational, document, key-value, and even graph-based data stores.

For instance, we can see in the diagram that the product catalog microservice consumes a relational database to accommodate the rich relational structure of its underlying data. On the other hand, the ordering microservice consumes both a NoSQL document database for write operations along with a highly denormalized key/value store to accommodate high volumes of read operations. Finally, the shopping cart microservice consumes a distributed cache that supports its simple, key-value data store.



While relational databases remain relevant for microservices with complex data, NoSQL databases have gained considerable popularity. They provide massive scale and high availability. Their schemaless nature allows developers to move away from an architecture of typed data classes and ORMs that make change expensive and time-consuming. We cover NoSQL databases later in this lesson.

This lesson explains cloud-native data patterns which refer to the methods and strategies employed to manage, store, and process data in cloud environments.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Compare relational vs. NoSQL data.

02 Describe Elasticsearch

03 Apply selection criteria to cloud database services.

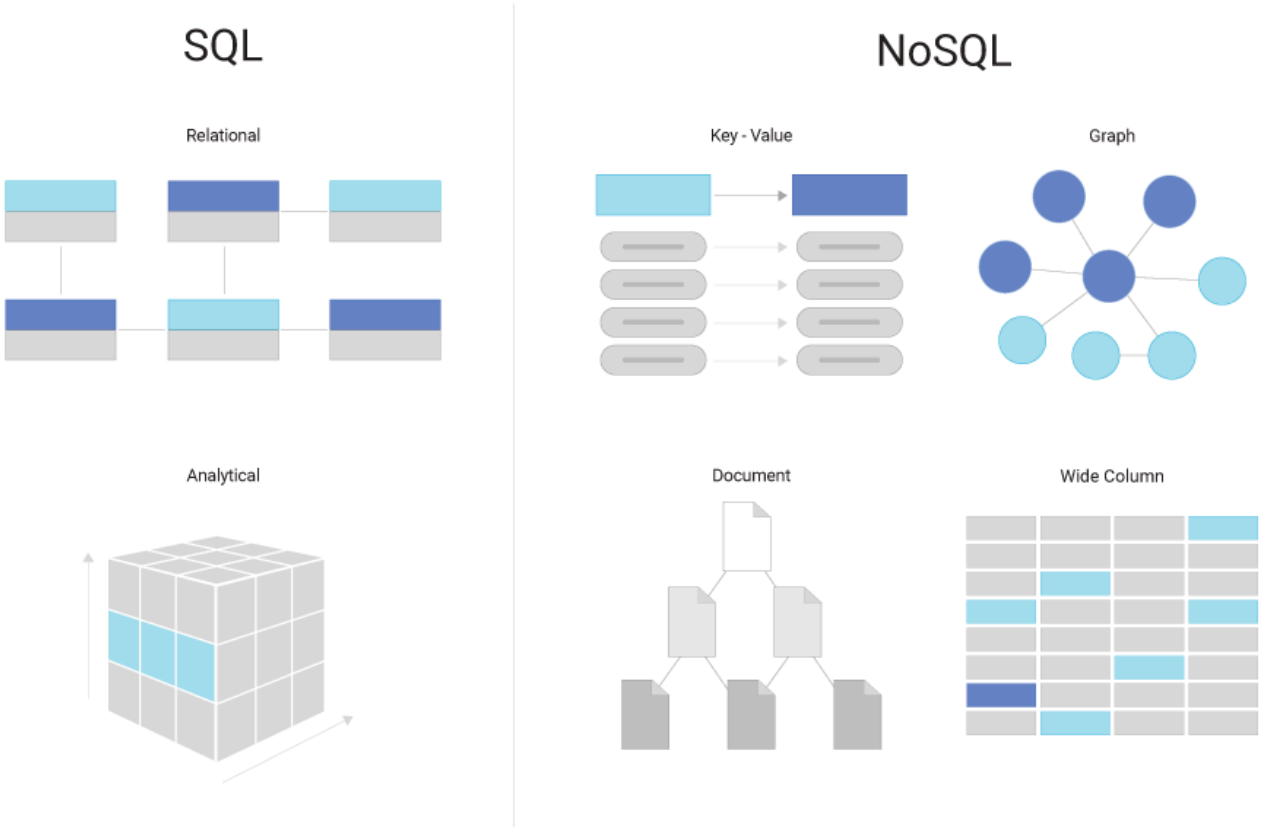
Key Terms and Concepts

Key Term or Concept	Definition
ACID Transaction	ACID is a set of properties of database transactions intended to guarantee data validity despite errors, power failures, and other mishaps. In the context of databases, a sequence of database operations that satisfies the ACID properties is called a transaction. ACID is an acronym that refers to the set of 4 key properties that define a transaction: Atomicity, Consistency, Isolation, and Durability. If a database operation has these ACID properties, it can be called an ACID transaction, and data storage systems that apply these operations are called transactional systems.

Key Term or Concept	Definition
Polyglot persistence	Polyglot persistence is a term that refers to using multiple data storage technologies within a single system, in order to meet varying data storage needs. Such a system may consist of multiple applications, or it may be a single application with smaller components.
Sharding	Sharding is a technique to distribute large amounts of identically structured data across a number of independent databases. It is especially popular with cloud developers creating Software as a Service (SAAS) offerings for end customers or businesses. These end customers are often referred to as "tenants". Sharding may be required for any number of reasons: • The total amount of data is too large to fit within the constraints of an individual database. • The transaction throughput of the overall workload exceeds the capabilities of an individual database. • Tenants may require physical isolation from each other, so separate databases are needed for each tenant. • Different sections of a database may need to reside in different geographies for compliance, performance, or geopolitical reasons.

Relational vs. NoSQL data

Relational and NoSQL are two types of database systems commonly implemented in cloud-native apps. They're built differently, store data differently, and are accessed differently.

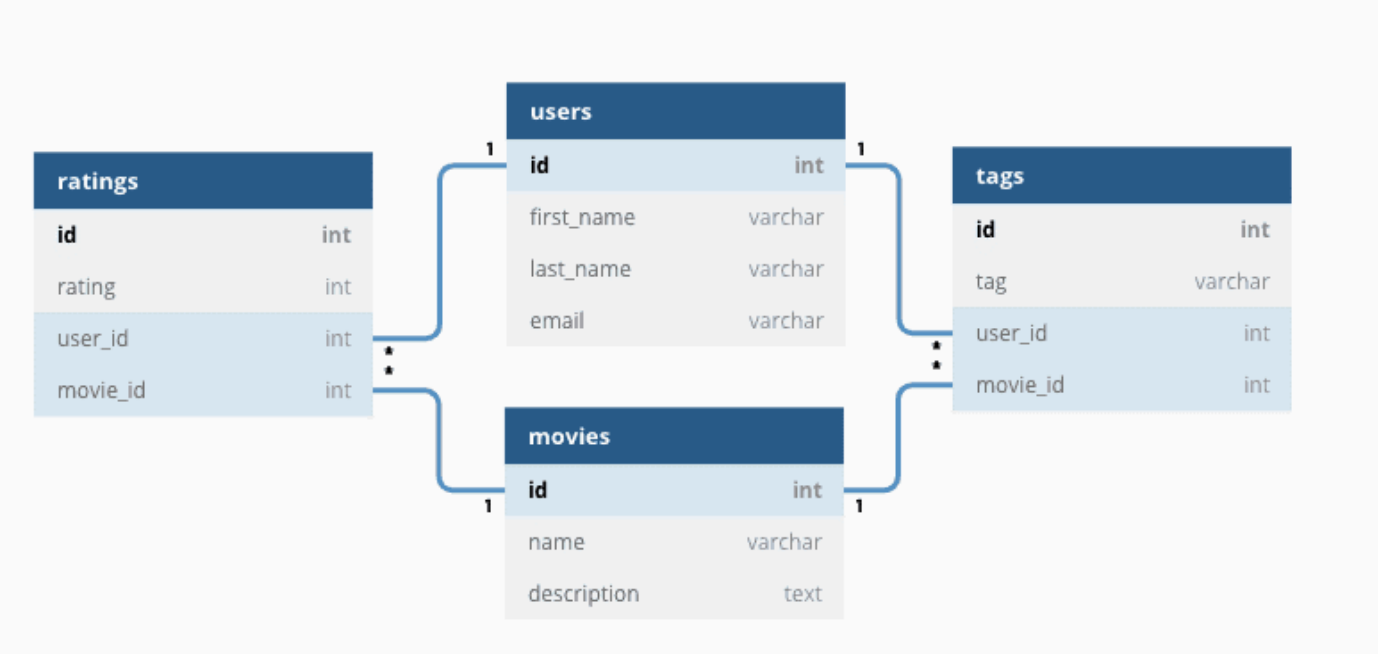


This section will provide an understanding of both implementations.

Relational databases

Relational databases have been around for a long time. They're mature, proven, and widely implemented. Competing database products, tooling, and expertise abound.

Relational databases provide a store of related data tables. These tables have a fixed schema, use SQL (Structured Query Language) to manage data, and support ACID guarantees.

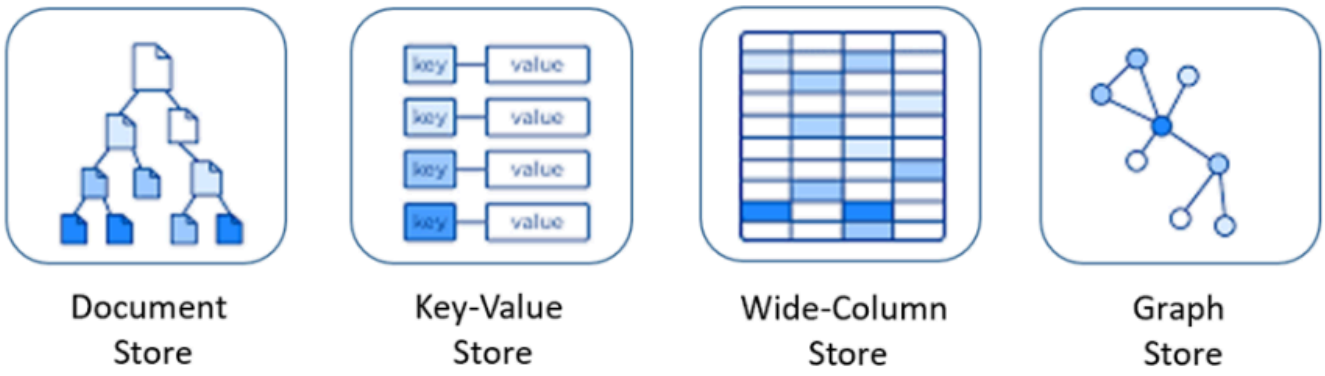


NoSQL databases

NoSQL databases typically don't provide ACID guarantees beyond the scope of a single database partition. High volume services that require sub second response time favor NoSQL datastores.

The impact of NoSQL technologies for distributed cloud-native systems can't be overstated. The proliferation of new data technologies in this space has disrupted solutions that once exclusively relied on relational databases.

NoSQL databases include several different models for accessing and managing data, each suited to specific use cases. Here are four common models used in NoSQL.



- **Document Store**: Data and metadata are stored hierarchically in JSON-based documents inside the database.
- **Key Value Store**: The simplest of the NoSQL databases, data is represented as a collection of key-value pairs.

- **Wide-Column Store**: Related data is stored as a set of nested-key/value pairs within a single column.
- **Graph Store**: Data is stored in a graph structure as node, edge, and data properties.

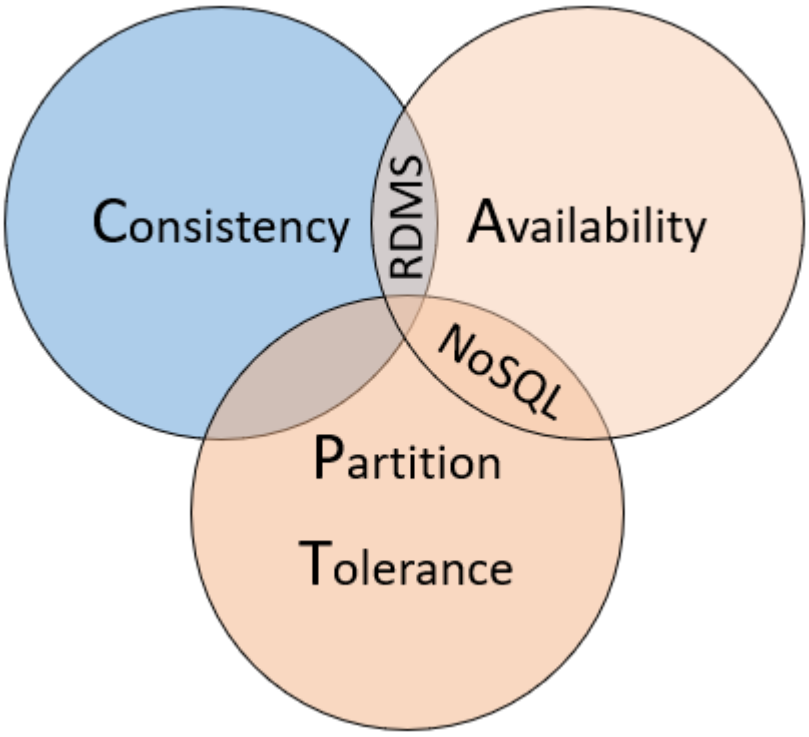
NoSQL databases have a very different approach to data management:

- They use high-performance, non-relational data stores.
- They excel in their ease-of-use, scalability, resilience, and availability characteristics.

Instead of joining tables of normalized data, NoSQL stores unstructured or semi-structured data, often in key-value pairs or JSON documents.

The CAP theorem

As a way to understand the differences between these types of databases, consider the CAP theorem, a set of principles applied to distributed systems that store state.



The theorem states that distributed data systems will offer a trade-off between consistency, availability, and partition tolerance. Further, it states that any database can only guarantee two of the three properties:

- Consistency: Every node in the cluster responds with the most recent data, even if the system must block the request until all replicas synchronize. If you query a "consistent system" for an item that is currently updating, you'll wait for that response until all replicas successfully update. However, you'll receive the most current data.

- Availability: Every node returns an immediate response, even if that response isn't the most recent data. If you query an "available system" for an item that is updating, you'll get the best possible answer the service can provide at that moment.

- Partition Tolerance: Guarantees the system continues to operate even if a replicated data node fails or loses connectivity with other replicated data nodes.

CAP theorem explains the trade-offs associated with managing consistency and availability during a network partition; however, trade-offs with respect to consistency and performance also exist with the absence of a network partition. CAP theorem is often further extended to PACELC to explain the trade-offs more comprehensively.

Relational databases typically provide consistency and availability, but not partition tolerance. They're typically provisioned to a single server and scale vertically by adding more resources to the machine.

Many relational database systems support built-in replication features where copies of the primary database can be made to other secondary server instances. Write operations are made to the primary instance and replicated to each of the secondaries. Upon a failure, the primary instance can fail over to a secondary to provide high availability. Secondaries can also be used to distribute read operations. While writes operations always go against the primary replica, read operations can be routed to any of the secondaries to reduce system load.

Data can also be horizontally partitioned across multiple nodes, such as with sharding. But, sharding dramatically increases operational overhead by spitting data across many pieces that cannot easily communicate. It can be costly and time consuming to manage. Relational features that include table joins, transactions, and referential integrity require steep performance penalties in sharded deployments.

Replication consistency and recovery point objectives can be tuned by configuring whether replication occurs synchronously or asynchronously. If data replicas were to lose network connectivity in a "highly consistent" or synchronous relational database cluster, you wouldn't be able to write to the database. The system would reject the write operation as it can't replicate that change to the other data replica. Every data replica must update before the transaction can complete.

NoSQL databases typically support high availability and partition tolerance. They scale out horizontally, often across commodity servers. This approach provides tremendous availability, both within and across geographical regions at a reduced cost. You partition and replicate data across these machines, or nodes, providing redundancy and fault tolerance. Consistency is typically tuned through consensus protocols or quorum mechanisms. They provide more control when navigating trade-offs between tuning synchronous versus asynchronous replication in relational systems.

If data replicas were to lose connectivity in a "highly available" NoSQL database cluster, you could still complete a write operation to the database. The database cluster would allow the write operation and update each data replica as it becomes available. NoSQL databases that support

multiple writable replicas can further strengthen high availability by avoiding the need for failover when optimizing recovery time objective.

Modern NoSQL databases typically implement partitioning capabilities as a feature of their system design. Partition management is often built-in to the database, and routing is achieved through placement hints - often called partition keys. A flexible data models enables the NoSQL databases to lower the burden of schema management and improve availability when deploying application updates that require data model changes.

High availability and massive scalability are often more critical to the business than relational table joins and referential integrity. Developers can implement techniques and patterns such as Sagas, CQRS, and asynchronous messaging to embrace eventual consistency. Nowadays, care must be taken when considering the CAP theorem constraints. A new type of database, called NewSQL, has emerged which extends the relational database engine to support both horizontal scalability and the scalable performance of NoSQL systems.

Considerations for relational vs. NoSQL systems

Consider a relational database when:

- Your workload volume generally fits within thousands of transactions per second.
- Your data is highly structured and requires referential integrity.
- Relationships are expressed through table joins on normalized data models.
- You work with complex queries and reports.
- Data is typically centralized or can be replicated regions asynchronously.
- Your application will be deployed to large, high-end hardware.

On the other hand, consider a NoSQL datastore when:

- You have high volume workloads that require predictable latency at large scale (e.g., latency measured in milliseconds while performing millions of transactions per second).
- Your data is dynamic and frequently changes.
- Relationships can be de-normalized data models.
- Data retrieval is simple and expressed without table joins.
- Data is typically replicated across geographies and requires finer control over consistency,

availability, and performance.

- Your application will be deployed to commodity hardware, such as with public clouds.

Database as a Service

Cloud-native applications favour data services exposed as a Database as a Service (DBaaS). Fully managed by a cloud vendor, these services provide built-in security, scalability, and monitoring. Instead of owning the service, you simply consume it as a backing service. The provider operates the resource at scale and bears the responsibility for performance and maintenance.

Authentication frameworks

There are a lot of solutions available when it comes to implementing an identity and access management system for an application. In this section, we discover some of these frameworks.

OAuth

An open standard that is currently at it second version. OAuth 2.0 is not an authentication protocol. It was created to securely delegate authorizations to other applications. It's a standard that provides limited access to an online service. Once you have proven your identity, OAuth can give third parties what is called "secure delegated access" to perform tasks on your behalf.

In this standard, there are four actors:

- Resource owner: the user who wants some work done on their behalf.
- Resource server: the online service where the work will be accomplished.
- Client: the application that will do the task for the resource owner
- Authorization server: the core of OAuth, that after verifying the identity of the resource owner, gives the client "access tokens".

The tokens remove the need to share passwords. The client receives a token that is encoded with the information required to prove to the resource server that it is authorized to perform the task. Tokens are not an all or nothing proposition, like a password would be, and are defined with a specific scope of action.

The Open Authorization standard, OAuth 2.0, is an authorization protocol developed by Okta. Read their explanation of the protocol here:

WHAT IS OAUTH 2.0? (HTTPS://AUTH0.COM/INTRO-TO-IAM/WHAT-IS-OAUTH-2)

This disseminated infrastructure scales well and is a good fit for cloud-native applications.

SAML

SAML stands for Security Assertion Markup Language and is a standardized way to transmit the level assurance that a user is the authentic individual. Like OAuth, SAML is currently in its second version.

The intent behind SAML is to increase the interoperability of the authentication process. Traditionally, an authentication process must be conducted for every system that is going to be used. The results are not transferable from one system to the other. SAML aims to create a system where the assurance can be used in multiple other systems after it has been established. This approach makes SAML a central component to a single sign on mechanism, known as SSO.

There are three components required for SAML:

- The principal, or the subject: the individual that wishes to authenticate to and use online systems.
- Identity provider: the service that stores information about the user and is capable of authenticating a user's identity. The identity provider grants the assertion to the user's identity.
- Service provider: this is the online system that the principal wishes to use.

The typical workflow with SAML is the following:

1. The principal requests a service from the service provider.
2. The service provider makes a request to the identity provider to establish the authenticity of the user.
3. The identity provider will either authenticate the user or use an existing authentication to assert the identity to the service provider.

Cloudfare provides a quick overview of how SAML works in the following article:

WHAT IS SAML? | HOW SAML AUTHENTICATION WORKS (HTTPS://WWW.CLOUDFLARE.COM/EN-CA/LEARNING/ACCESS-MANAGEMENT/WHAT-IS-SAML/)

OpenID Connect

OpenID Connect is an interoperable authentication protocol based on the OAuth 2.0 framework of specifications. It simplifies the way to verify the identity of users based on the authentication performed by an Authorization Server and to obtain user profile information in an interoperable and REST-like manner.

OpenID Connect enables application and website developers to launch sign-in flows and receive verifiable assertions about users across Web-based, mobile, and JavaScript clients. And the specification suite is extensible to support a range of optional features such as encryption of identity data, discovery of OpenID Providers, and session logout.

For developers, it provides a secure and verifiable answer to the question “What is the identity of the person currently using the browser or mobile app that is connected?” Best of all, it removes the responsibility of setting, storing, and managing passwords which is frequently associated with credential-based data breaches.

The following resource provide a humoristic explanation of how OpenID Connect works:

OPENID EXPLAINED (HTTP://OPENIDEXPLAINED.COM/)

Fast ID Online (FIDO)

Fast ID online, or FIDO2, is the evolution of FIDO Universal Second Factor (U2F) that removes passwords from the authentication process in favour of hardware tokens. The overall objective for FIDO2 is to provide an extended set of functionalities to cover additional use-cases, with the main driver being passwordless login flows. The U2F model is still the basis for FIDO2 and compatibility for existing U2F deployments is provided in the FIDO2 specs.

FIDO2 is the protocol behind the Yubikey security hardware token.


enter
shift
J
K
L
N
M
alt
ctrl
pg
pg
intel
CORE
vPRO
i5

Identity Management

Software applications require knowledge of the user or process calling them. The user/process is called a security principal, and the mechanism to define, authorize and authenticate principals is called identity management.

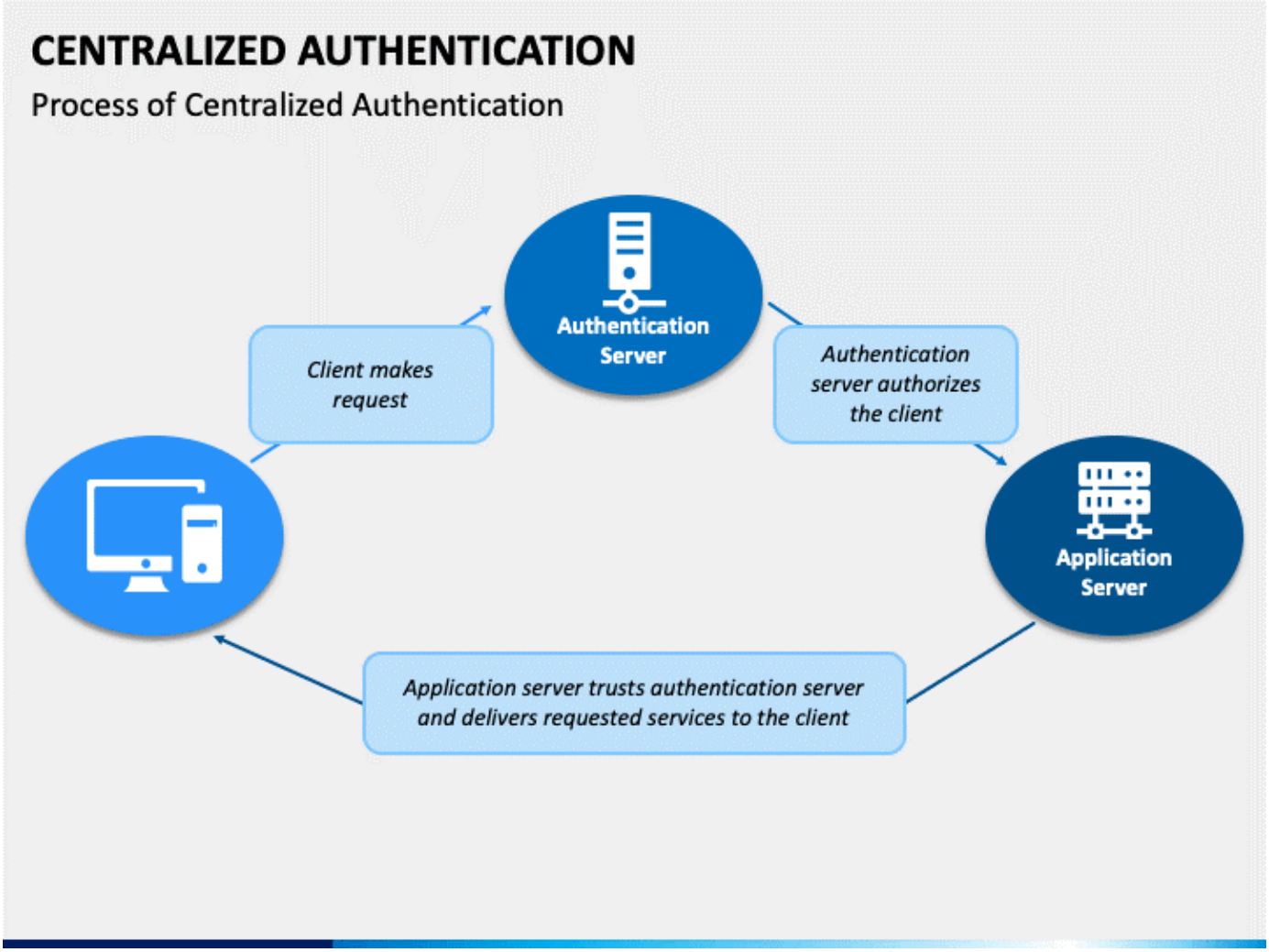
Simple apps may have all identity management within, but it doesn't scale well when the number of apps and security principals increases.



Centralized Authentication and Authorization

Using centralized authentication and authorization mechanisms allows to manage user access to resources from a single point of control.

Active Directory or similar directory services are used to authenticate users and manage their permissions across the network. Users authenticate themselves with credentials, and the central system verifies their identity and grants access if authorized.



The benefits of centralized authentication and authorization include:

- simplified user management
- improved security
- consistent enforcement of security policies

It also provides a central point of control for security administration and reduces the overhead of managing user access.

Active Directory

Windows supports the use of Active Directory to provide centralized authentication and authorization.

While this solution is effective within corporate networks, it isn't designed for use by users or applications that are outside of the AD domain.

With the growth of Internet-based applications and the rise of cloud-native apps, security models have evolved.

Microsoft Active Directory

Cloud-Native Identity Management

Cloud-native identity assumes a distributed architecture. This means that apps can be deployed and communicate anywhere. Clients can also communicate from anywhere and on any platform or device.

Cloud-native identity solutions use open standards for secure application access. Clients can range from humans on PCs or phones to apps, set-top boxes, and IoT devices running any software platform anywhere in the world.

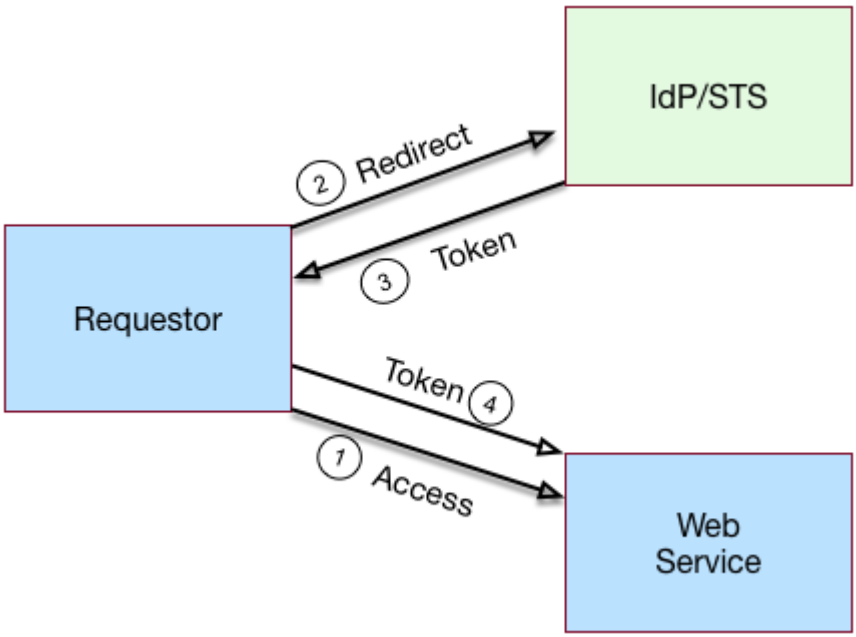


Token-based authentication

Modern cloud-native identity solutions typically use access tokens that are issued by a secure token service/server (STS) to a security principal once their identity is determined.

The access token, typically a JSON Web Token (JWT), includes claims about the security principal, including the user's identity and other claims that determine the level of access to grant to the principal.

the STS is only responsible for authenticating the principal. Determining their level of access to resources is left to other parts of the application.

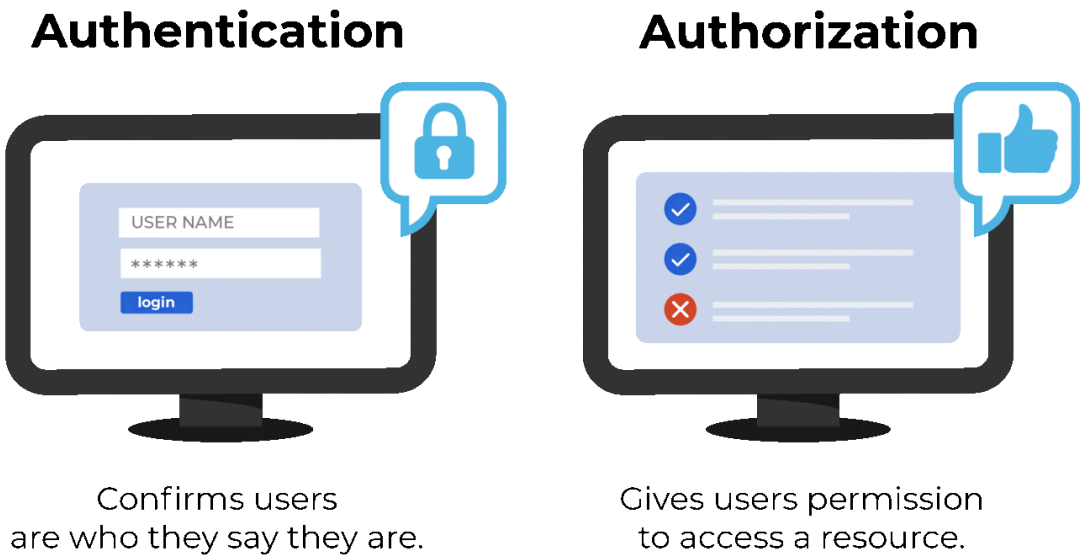


Authentication and authorization in cloud-native apps

Cloud-native applications need to rely on open HTTP-based protocols to authenticate security principals since both clients and applications could be running anywhere in the world on any platform or device. The only common factor is HTTP.

Authentication (AuthN) is the process of establishing a proper level of assurance towards the identity of a security principal.

Authorization (AuthZ) is the act of granting an authenticated principal permission to perform an action or access a resource.



Cloud-based identity and access management services

Cloud service providers offer identity and access management services to their customers. Here is a short list of some of the services available:

- **Okta**: cloud-based platform for identity management, authentication, authorization, and user management.
- **Google Cloud Identity**: cloud-based service for identity and access management, offering authentication, authorization, and user provisioning.
- **AWS IAM**: cloud-based service for authentication, authorization, and user management in AWS.
- **Microsoft Azure AD**: cloud-based service for authentication and authorization in cloud-based apps and resources.

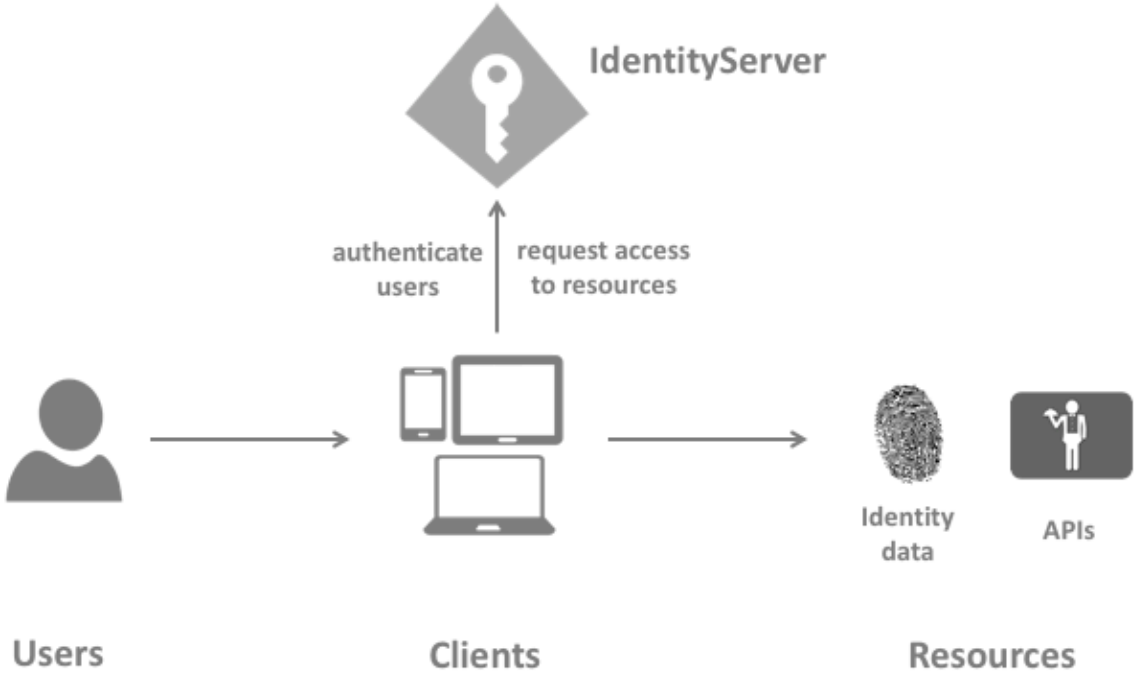
Token-based authentication is an interesting mechanism that provides versatility and utility to access management requirements. The following article from Okta provides an overview of the mechanism:

WHAT IS TOKEN-BASED AUTHENTICATION? (HTTPS://WWW.OKTA.COM/IDENTITY-101/WHAT-IS-TOKEN-BASED-AUTHENTICATION/)

IdentityServer

IdentityServer is an open-source identity and access management framework that can be used for cloud-native applications. It provides authentication and authorization services using various standard protocols, such as OAuth 2.0, OpenID Connect, and SAML.

IdentityServer can be integrated with various programming languages and platforms, including .NET, Java, and JavaScript. It also offers features such as user authentication, role-based authorization, and token management.

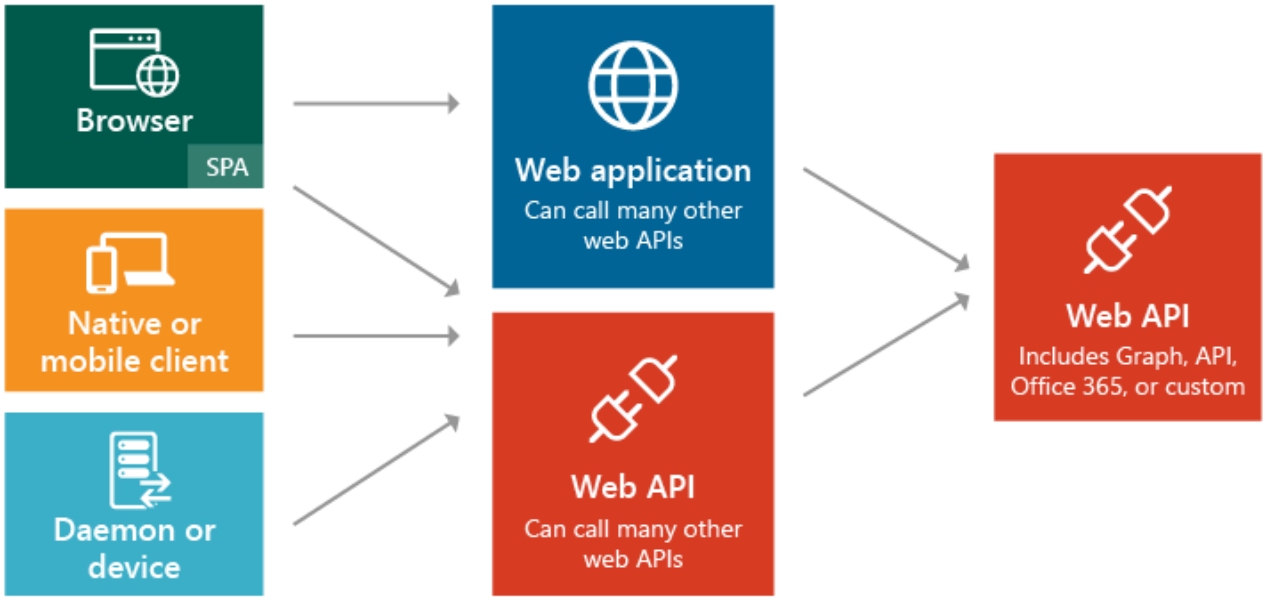


It can simplify the implementation of authentication and authorization in cloud-native applications while providing a high degree of security and flexibility.

Common web app scenarios

Typically, applications need to support some or all of the following scenarios:

- Human users accessing web applications with a browser.
- Human users accessing back-end Web APIs from browser-based apps.
- Human users on mobile/native clients accessing back-end Web APIs.
- Other applications accessing back-end Web APIs (without an active user or user interface).



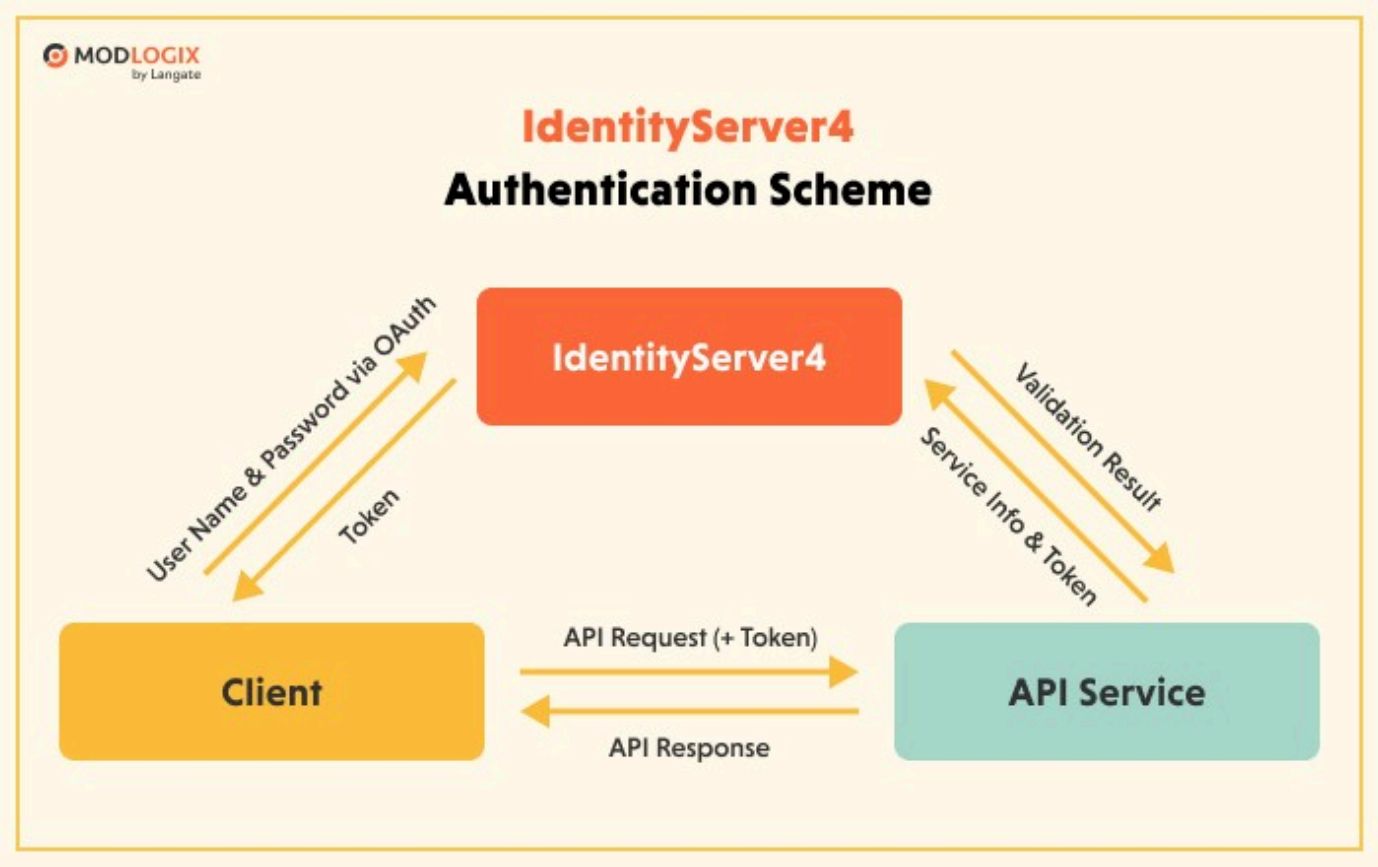
Any application may need to interact with other Web APIs, using its own identity or delegating to the user's identity.

In each of these scenarios, the exposed functionality needs to be secured against unauthorized use. At a minimum, this typically requires authenticating the user or principal making a request for a resource. This authentication may use one of several common protocols such as SAML2p, WS-Fed, or OpenID Connect. Communicating with APIs typically uses the OAuth2 protocol and its support for security tokens. Separating these critical cross-cutting security concerns and their implementation details from the applications themselves ensures consistency and improves security and maintainability. Outsourcing these concerns to a dedicated product like IdentityServer helps the requirement for every application to solve these problems itself.

IdentityServer for cloud-native applications

IdentityServer provides middleware that runs within an ASP.NET Core application and adds support for OpenID Connect and OAuth2.

Organizations would create their own ASP.NET Core app using IdentityServer middleware to act as the STS for all of their token-based security protocols.



The IdentityServer middleware exposes endpoints to support standard functionality, including:

- Authorize (authenticate the end user)
- Token (request a token programmatically)
- Discovery (metadata about the server)
- User Info (get user information with a valid access token)
- Device Authorization (used to start device flow authorization)
- Introspection (token validation)
- Revocation (token revocation)
- End Session (trigger single sign-out across all apps)

This lesson will explain cloud-native identity which refer to the approach of managing identities in a cloud-native environment that uses access tokens issued by a secure token service/server

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Recall the basics of identity management for cloud infrastructure.

02 Define how centralized authentication and authorization works.

03 Recall common cloud-based identity and access management services.

Key Terms and Concepts

Key Term or Concept	Definition
Middleware	Middleware is software that different applications use to communicate with each other. It provides functionality to connect applications intelligently and efficiently so that you can innovate faster. Middleware acts as a bridge between diverse technologies, tools, and databases so that you can integrate them seamlessly into a single system.

Learning Activity



Provide answers to the following questions:

1. How do SAML, OpenID, and OAuth resemble each other?

Click to see the answer.

2. How does the authentication mechanism manage to provide interoperability between vendors?

Click to see the answer.

3. What is IdentityServer?

Click to see the answer.

4. What is the Service Provider in SAML?

Click to see the answer.

5. What is the difference between authentication and authorization?

Click to see the answer.

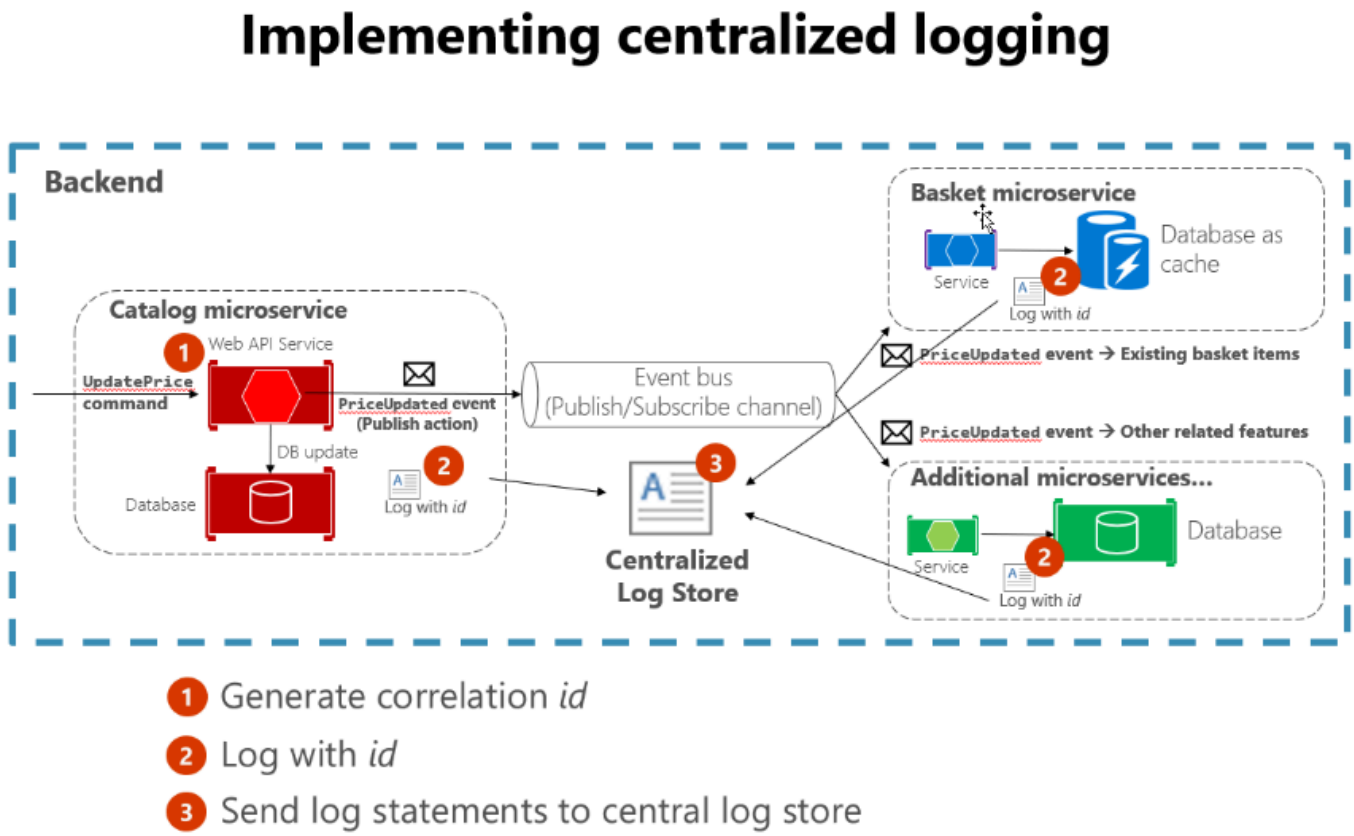
H-P Reuse I am confused

Centralized logging

Because of the challenges associated with using file-based logs in cloud-native apps, centralized logs are preferred. Logs are collected by the applications and shipped to a central logging application which indexes and stores the logs. This class of system can ingest tens of gigabytes of logs every day.

Following standard practices is useful when building logging for multiple services. Generating a correlation ID at the start of an interaction and logging it in related messages makes it easier to search for all related messages. Ensuring that the log format is the same for every service, regardless of the language or logging library used, makes reading logs much easier. Centralized logging can be leveraged in a microservices architecture to streamline the logging workflow.

It's also helpful to follow some standard practices when building logging that spans many services. For instance, generating a correlation ID at the start of a lengthy interaction, and then logging it in each message that is related to that interaction, makes it easier to search for all related messages. One need only find a single message and extract the correlation ID to find all the related messages. Another example is ensuring that the log format is the same for every service, whatever the language or logging library it uses. This standardization makes reading logs much easier. The following figure demonstrates how a microservices architecture can leverage centralized logging as part of its workflow.



Monitoring cloud-native apps

Centralized logging systems can collect metrics in addition to logs, providing a holistic view of the health of nodes and the application. Monitoring tools can be fed with metric data manually from within the application, allowing for the monitoring of business flows.

Queries can be constructed in log aggregation tools to look for specific statistics or patterns, which can be displayed in custom dashboards. Cloud-native monitoring tools provide real-time telemetry and insight into applications, regardless of whether they're monolithic or microservices architectures. These tools include data collection, querying, and display features for monitoring an application's health.

Challenges with reacting to critical problems in cloud-native apps

Alerting is the third cloud-native application observability pattern and relies on logging and monitoring. Logging is necessary to diagnose problems and can feed into monitoring tools.

Monitoring aggregates metrics and health data in one place, allowing rules to be created that trigger alerts when certain metrics fall outside of acceptable levels. Alerts are layered on top of monitoring, with certain conditions triggering appropriate alerts to notify team members of urgent problems.

Scenarios that may require alerts include unresponsive services, unsuccessful HTTP responses, and slow response times for key endpoints.

Challenges with detecting and responding to potential app health issues Some applications aren't mission critical. Maybe they're only used internally, and when a problem occurs, the user can contact the team responsible, and the application can be restarted. However, customers often have higher expectations for the applications they consume. You should know when problems occur with your application before users do, or before users notify you. Otherwise, the first you know about a problem may be when you notice an angry deluge of social media posts deriding your application or even your organization. Some scenarios you may need to consider include: One service in your application keeps failing and restarting, resulting in intermittent slow responses. At some times of the day, your application's response time is slow. After a recent deployment, load on the database has tripled. Implemented properly, monitoring can let you know about conditions that will lead to problems, letting you address underlying conditions before they result in any significant user impact.

Alerts in cloud-native apps

Queries can be crafted against monitoring tools to look for known failure conditions, such as HTTP status code 500. Alerts can be sent via email or SMS to the owner of the originating service when a problem is detected. Alert queries can be crafted to only fire when a larger than average number of errors are detected to avoid false positives.



Firing too many alerts can desensitize service owners and lead to true errors being lost in the noise of false positives. It's important to ensure that alerts are indicative of a real problem to avoid this issue.

Some centralized logging systems take on an additional role of collecting telemetry outside of pure logs. They can collect metrics, such as time to run a database query, average response time from a web server, and even CPU load averages and memory pressure as reported by the operating system. In conjunction with the logs, these systems can provide a holistic view of the health of nodes in the system and the application as a whole.

The metric-gathering capabilities of the monitoring tools can also be fed manually from within the application. Business flows that are of particular interest such as new users signing up or orders being placed, may be instrumented such that they increment a counter in the central monitoring system. This aspect unlocks the monitoring tools to not only monitor the health of the application but the health of the business.

Queries can be constructed in the log aggregation tools to look for certain statistics or patterns, which can then be displayed in graphical form, on custom dashboards. Frequently, teams will invest in large, wall-mounted displays that rotate through the statistics related to an application. This way, it's simple to see the problems as they occur.

Cloud-native monitoring tools provide real-time telemetry and insight into apps regardless of whether they're single-process monolithic applications or distributed microservice architectures. They include tools that allow collection of data from the app as well as tools for querying and displaying information about the app's health.

This lesson will explain health and monitoring which are critical aspects of cloud-native applications that help ensure their availability and performance. Health monitoring involves periodically checking the application's health status, including its availability, responsiveness, and resource utilization.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Recall the basis of observability patterns.

02 Implement monitoring and health.

Learning Activity



Consider the following questions and provide answers for each:

1. What are the three observability patterns?

Click to see the answer.

2. How can logs be useful to developers?

Click to see the answer.

3. What can be the issue with a file-based logging system?

Click to see the answer.

4. What are some of the typical severity levels used by log standards?

Click to see the answer.

H-P Reuse I am confused

Logging with Elastic Stack

There are many good centralized logging tools and they vary in cost from being free, open-source tools, to more expensive options. In many cases, the free tools are as good as or better than the paid offerings.

One such tool is a combination of three open-source components: Elasticsearch, Logstash, and Kibana. Collectively these tools are known as the Elastic Stack or ELK stack.



Elastic Stack provides centralized logging in a low-cost, scalable, cloud-friendly manner. Its user interface streamlines data analysis so you can spend your time gleaning insights from your data instead of fighting with a clunky interface. It supports a wide variety of inputs so as your distributed application spans more and different kinds of services, you can expect to continue to be able to feed log and metric data into the system. The Elastic Stack also supports fast searches even across large data sets, making it possible even for large applications to log detailed data and still be able to have visibility into it in a performant fashion.

Logstash

The first component is Logstash. This tool is used to gather log information from a large variety of different sources. For instance, Logstash can read logs from disk and also receive messages from logging libraries. Logstash can do some basic filtering and expansion on the logs as they arrive. For instance, if your logs contain IP addresses then Logstash may be configured to do a geographical lookup and obtain a country/region or even city of origin for that message.

Elasticsearch

Elasticsearch is a powerful search engine that can index logs as they arrive. It makes running queries against the logs quick. Elasticsearch can handle huge quantities of logs and, in extreme cases, can be scaled out across many nodes.

Log messages that have been crafted to contain parameters or that have had parameters split from them through Logstash processing, can be queried directly as Elasticsearch preserves this information.

Visualizing information with Kibana web dashboards

The final component of the stack is Kibana. This tool is used to provide interactive visualizations in a web dashboard. Dashboards may be crafted even by users who are non-technical. Most data that is resident in the Elasticsearch index, can be included in the Kibana dashboards. Individual users may have different dashboard desires and Kibana enables this customization through allowing user-specific dashboards.

Cloud-based monitoring and analytics service

Here are some of the cloud-based monitoring and analytics services available:

- **Azure Monitor**: provides full-stack observability for Azure and other cloud environments.
- **Google Cloud Monitoring**: monitors and logs Google Cloud Platform resources and applications.
- **AWS CloudWatch**: monitors and logs AWS resources and applications.

Each of these services collects telemetry data and provide visualization, alerting, and analysis features.


Google Cloud Platform
aws
Microsoft
Azure

Monitoring and health

Monitoring the health and efficiency of applications running in production is a key component of DevOps. This requires monitoring logs and metrics from servers, hosts, and applications. The many services supporting cloud-native applications can make monitoring challenging for individual components and the application as a whole.



In this lesson, we will analyze approaches and methods to monitor cloud-native applications.

Cloud-native and DevOps

Microservices and cloud-native applications go hand in hand with good DevOps practices. DevOps is many things to many people but perhaps one of the better definitions comes from cloud advocate and DevOps evangelist Donovan Brown:

"DevOps is the union of people, process, and products to enable continuous delivery of value to our end users.

Unfortunately, with terse definitions, there's always room to say more things. One of the key components of DevOps is ensuring that the applications running in production are functioning properly and efficiently. To gauge the health of the application in production, it's necessary to monitor the various logs and metrics being produced from the servers, hosts, and the application proper. The number of different services running in support of a cloud-native application makes monitoring the health of individual components and the application as a whole a critical challenge.

Observability patterns

Observability patterns are essential for maintaining reliable applications. Three key patterns that have emerged are:

- **Logging**: the process of collecting and storing log data that provides insights into the behavior of the application, including errors and system events.
- **Monitoring**: the process of collecting and analyzing metrics that provide insights into the performance and behavior of the application, such as CPU usage, memory usage, and network traffic.
- **Alerts**: the process of notifying stakeholders when specific conditions or thresholds are met, such as when errors occur, or performance metrics reach critical levels.



When to use logging

No matter how careful we are, applications almost always behave in unexpected ways in production. When users report problems with an application, it's useful to be able to see what was going on with the app when the problem occurred. One of the most tried and true ways of capturing information about what an application is doing while it's running is to have the application write down what it's doing. This process is known as logging. Anytime failures or problems occur in production, the goal should be to reproduce the conditions under which the failures occurred, in a non-production environment.

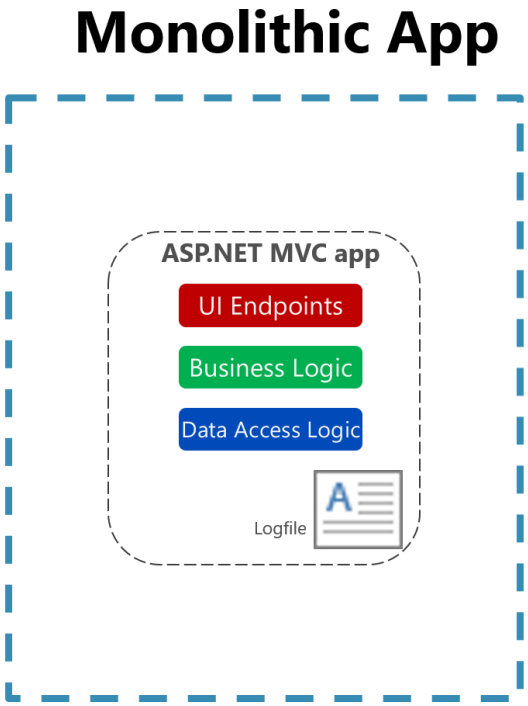


Having good logging in place provides a roadmap for developers to follow in order to duplicate problems in an environment that can be tested and experimented with.

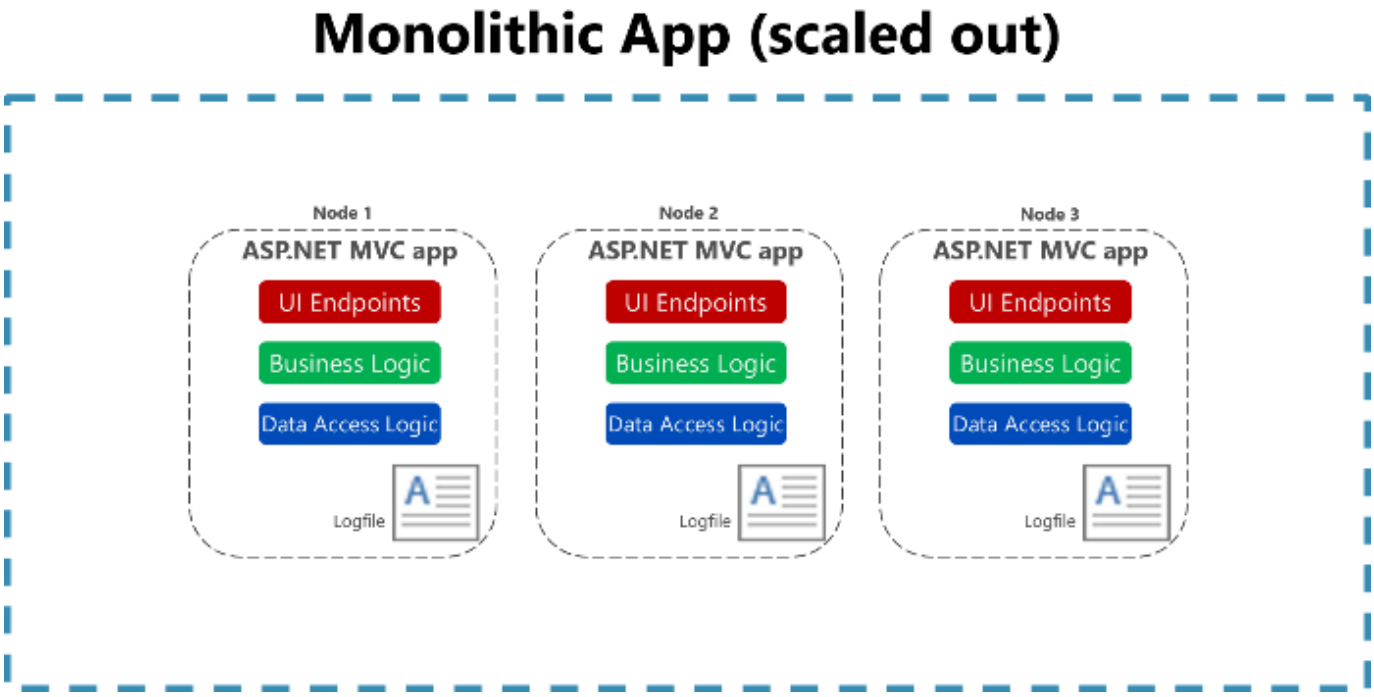
Logging is useful for capturing information about an application's behavior while it's running. It is particularly useful for diagnosing unexpected problems that occur in production environments. Good logging can help operators to reproduce problems in non-production environments and diagnose and fix issues effectively.

Challenges when logging with cloud-native applications

In traditional applications, log files are typically stored on the local machine. In fact, on Unix-like operating systems, there's a folder structure defined to hold any logs, typically under /var/log.

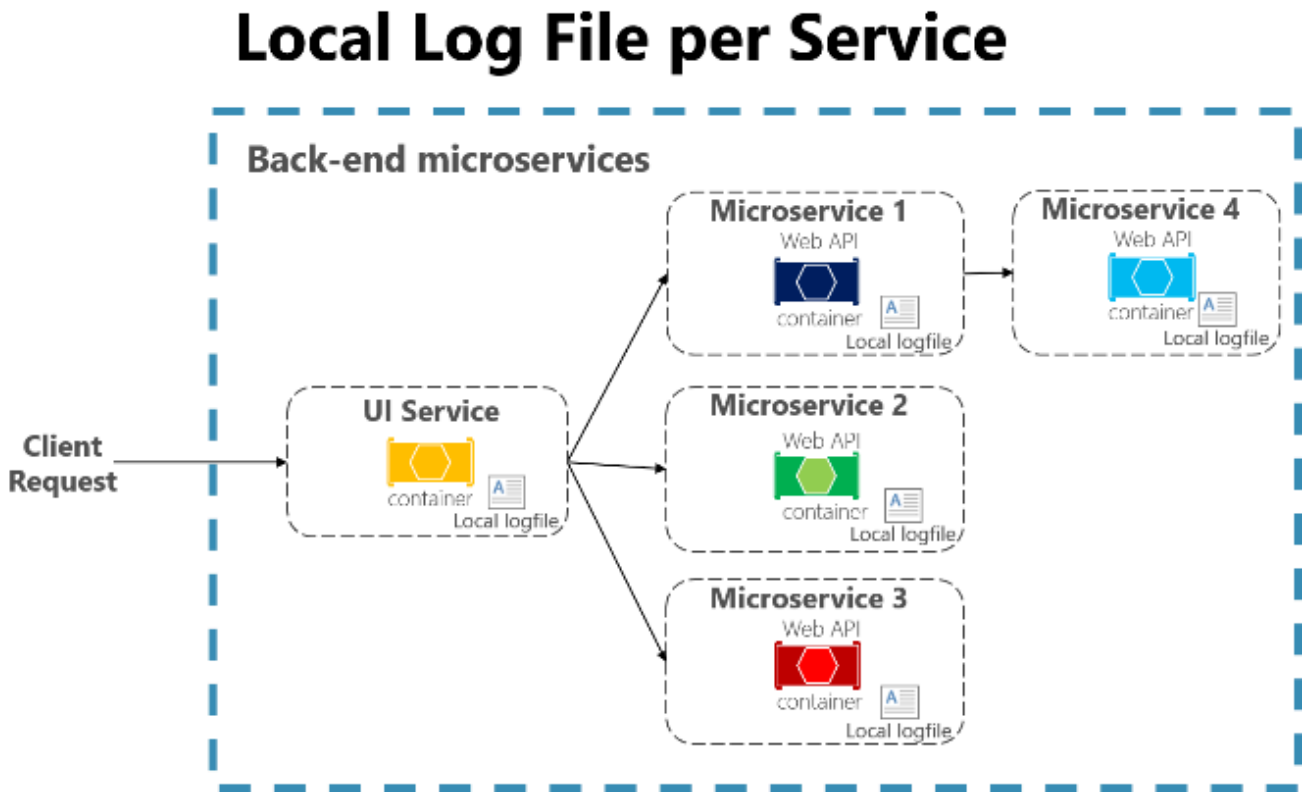


The usefulness of logging to a flat file on a single machine is vastly reduced in a cloud environment. Applications producing logs may not have access to the local disk or the local disk may be highly transient as containers are shuffled around physical machines. Even simple scaling up of monolithic applications across multiple nodes can make it challenging to locate the appropriate file-based log file.



Cloud-native applications developed using a microservices architecture also pose some challenges for file-based loggers. User requests may now span multiple services that are run on different machines and may include serverless functions with no access to a local file system at all. It would be very challenging to correlate the logs from a user or a session across these many services and machines.

The number of users in some cloud-native applications is high. Imagine that each user generates a hundred lines of log messages when they log into an application. In isolation, that is manageable, but multiply that over 100,000 users and the volume of logs becomes large enough that specialized tools are needed to support effective use of the logs.



Logging in cloud-native applications

Every programming language has tooling that permits writing logs, and typically the overhead for writing these logs is low. Many of the logging libraries provide logging different kinds of criticalities, which can be tuned at run time. For instance, syslog, a popular standard for message logging, provides different severity levels for log entries that include:

- Debug
- Information
- Notice
- Warning
- Error
- Critical
- Alert
- Emergency

These different log levels provide granularity in logging. When the application is functioning properly in production, it may be configured to only log important messages. When the application is misbehaving, then the log level can be increased so more verbose logs are gathered. This balances performance against ease of debugging.

The high performance of logging tools and the tunability of verbosity should encourage developers to log frequently. Many favour a pattern of logging the entry and exit of each method. This approach may sound like overkill, but it's infrequent that developers will wish for less logging. In fact, it's not uncommon to perform deployments for the sole purpose of adding logging around a problematic method. Err on the side of too much logging and not on too little. Some tools can be used to automatically provide this kind of logging.

The following resource provides a useful guide on the use of the syslog standard:

WHAT IS SYSLOG AND HOW DOES IT WORK? (HTTPS://WWW.AUVIK.COM/FRANKLYIT/BLOG/WHAT-IS-SYSLOG/)